VINUNIVERSITY



Ngày 26

MCP/A2A Infrastructure & Agentic Routing

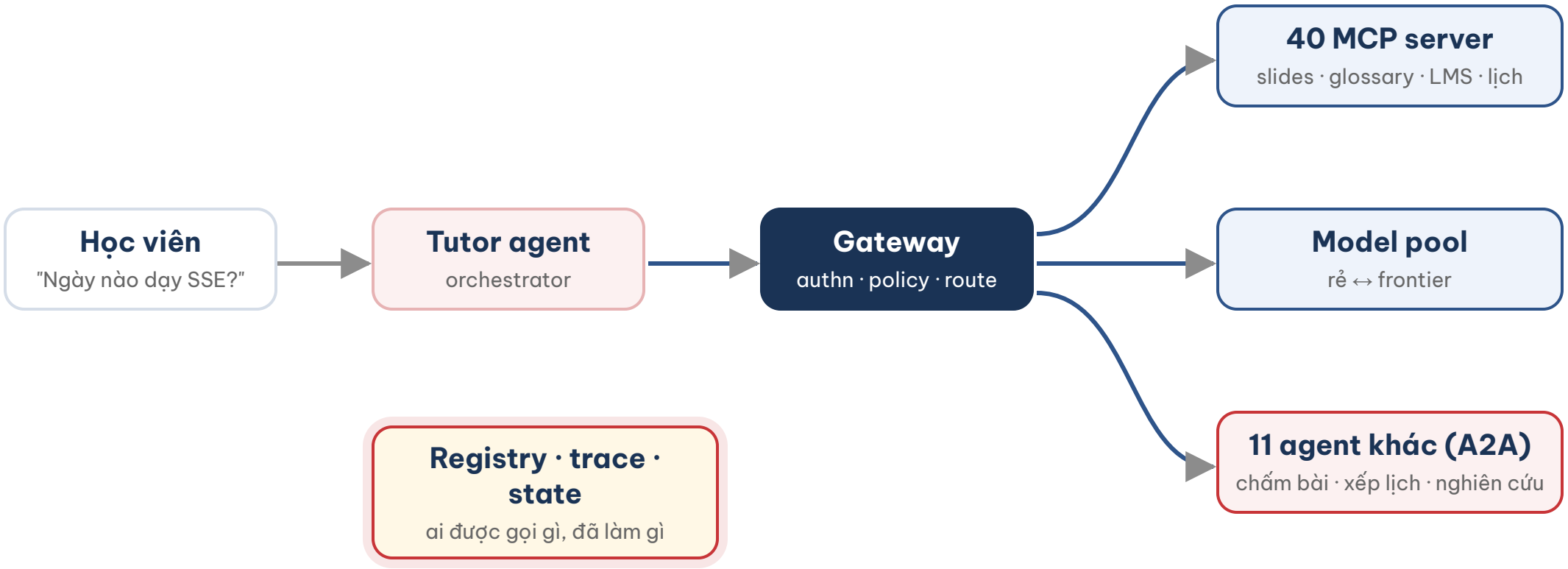
Vận hành 40 MCP server và 12 agent sau một gateway: định tuyến đúng, an toàn trên từng lời gọi.

GIẢNG VIÊN
[Tên giảng viên]

CẬP NHẬT
12/10/2026

THỜI LƯỢNG
4 giờ + lab

Hệ thống ta sẽ dựng trong đầu suốt buổi: VLearn Tutor



●●●●● Bên dưới là phần không ai nhìn thấy: registry, trace, trạng thái bền.

Minh hoạ · VLearn Tutor là ví dụ dựng cho bài học; cùng thế giới với lab Colosseum buổi chiều.

Bốn phần, mười một module, một câu hỏi xuyên suốt

VINUNIVERSITY

A · BỐI CẢNH

01 Lịch sử giao thức

02 MCP phi trạng thái

B · KIẾN TRÚC

03 Kiến trúc tham chiếu

04 Discovery, vòng đời

05 Gateway

06 Agentic routing

C · VẬN HÀNH

07 Danh tính, supply chain

08 Vòng lặp là workload

09 Quan sát xuyên agent

D · FRONTIER

10 Nghiên cứu mở

11 Lab

CÂU HỎI XUYÊN SUỐT

Hỏng thế nào khi có **40 cái**, không phải 1?

Hết buổi sáng, bạn làm được năm việc

01 Giải thích vì sao spec MCP `2026-07-28` là thay đổi **hạ tầng**.

02 Vẽ control plane và data plane của một agent platform.

03 Chọn kỹ thuật định tuyến model, tool, agent theo chi phí.

04 Đặt danh tính và policy lên **từng lời gọi**.

05 Scale vòng lặp agent theo đúng tín hiệu.

VINUNIVERSITY

01

Từ function calling đến hệ sinh thái giao thức

Mỗi chuẩn đóng một ranh giới tích hợp, và mở ra một bài toán hạ tầng.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Kể lại mốc chính 2023–2026 và vấn đề mỗi mốc giải quyết.

02 Đặt MCP, A2A, AG-UI, AP2 vào đúng cạnh của sơ đồ.

03 Phân biệt "chung quản trị" với "hợp nhất giao thức".

CÂU HỎI MỞ ĐẦU 10 ứng dụng AI, 40 tool. Không có chuẩn chung thì phải viết bao nhiêu bộ tích hợp?

Giao thức biến phép nhân thành phép cộng

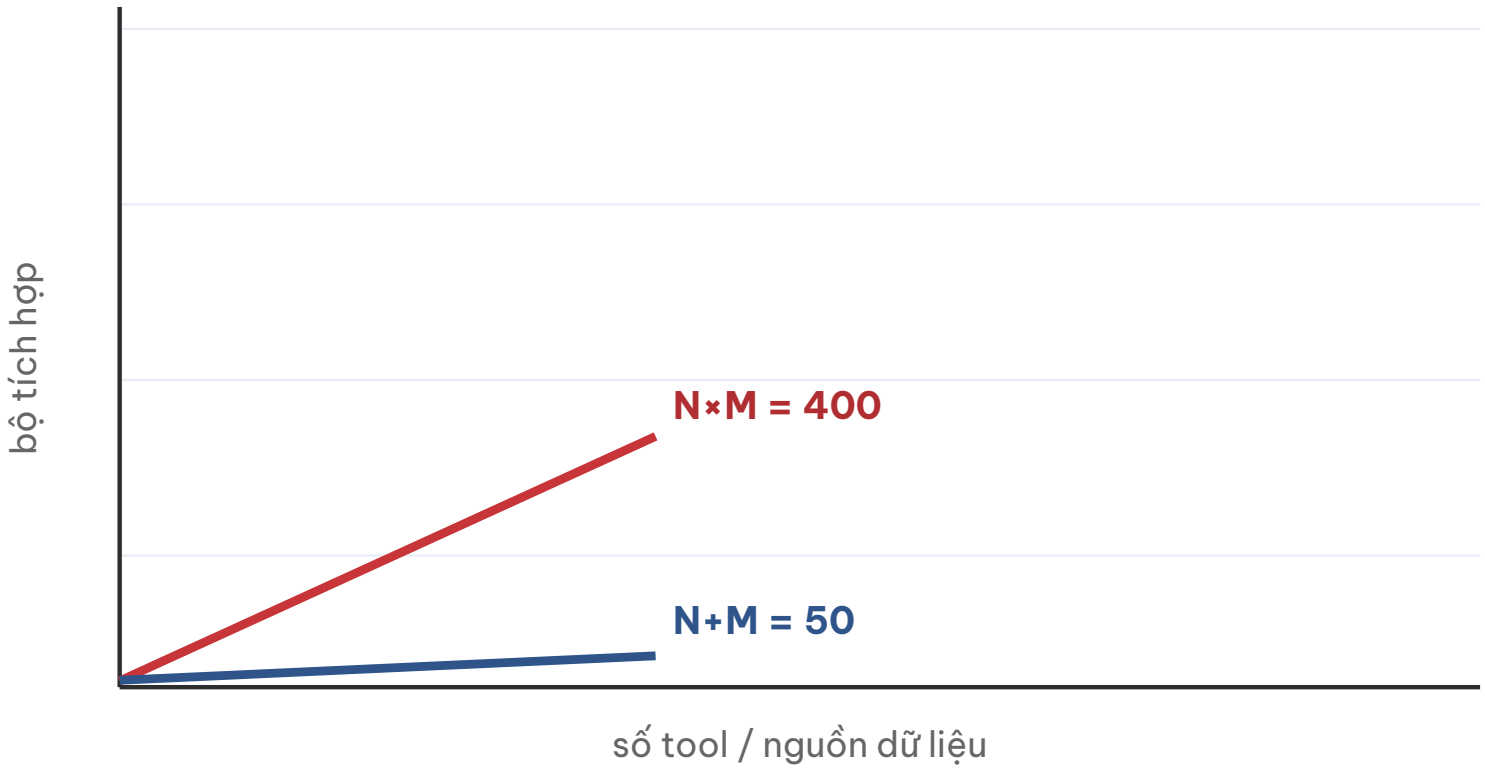
Ứng dụng AI 10

Tool / nguồn dữ liệu 40

400
bộ tích hợp riêng

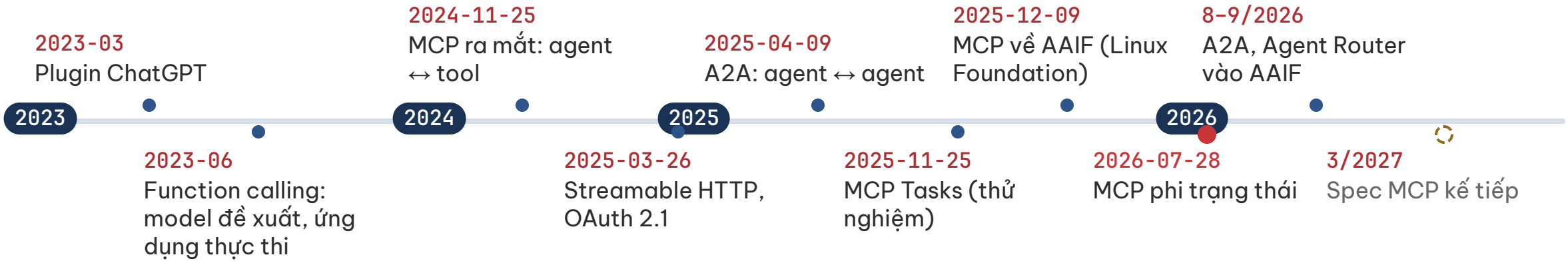
50
với một giao thức

DỰ ĐOÁN TRƯỚC
Nhấn → để hiện đường xanh. Khi nào hai đường gặp nhau?



Nguồn: modelcontextprotocol.io/docs/getting-started/intro · anthropic.com/news/model-context-protocol (2024-11-25)

Ba năm, năm làn sóng



●●●●● Bản kế tiếp lùi từ 15/12/2026 sang 3/2027, chu kỳ tháng 3 và tháng 9.

Nguồn: github.com/modelcontextprotocol/modelcontextprotocol/releases · discussions/3402 (họp 15/9/2026) · aaif.io/blog

Đoán trước: mỗi chuẩn để lại bài toán hạ tầng nào?

	Nó giải quyết	Nó để lại cho đội hạ tầng
Function calling	Model trả về lời gọi có cấu trúc	?
MCP	Một bề mặt chung agent ↔ tool	?
A2A	Giao việc giữa agent khác hãng	?
AP2 · ACP · UCP	Agent mua hàng, thanh toán	?

Lớp đoán từng hàng, rồi bấm để lộ.

Che lại

Nguồn: openai.com/index/function-calling-and-other-api-updates · a2a-protocol.org/v1.0.1 · cloud.google.com/blog (AP2, 2025-09-16)

Mỗi giao thức sở hữu một cạnh của sơ đồ

người ↔ agent	**AG-UI · MCP Apps**	luồng sự kiện tới giao diện
agent ↔ tool	**MCP**	tool, dữ liệu, resource
agent ↔ agent	**A2A**	giao task, theo dõi vòng đời
agent ↔ tiền	**AP2 · ACP · UCP**	ý định, giỏ hàng, thanh toán
xuyên suốt	**Hạ tầng**	registry · gateway · danh tính · trace

Lớp cuối không thuộc giao thức nào. Đó là phần của chúng ta hôm nay.

Nguồn: a2a-protocol.org (bài "A2A joins AAIF", 27/8/2026: MCP = vertical, A2A = horizontal) · blog.modelcontextprotocol.io (MCP Apps, 2026-01-26)

VINUNIVERSITY

Tutor cần agent chấm bài của đội khác chấm 200 bài, mất 2 giờ. Dùng gì?

A Gọi agent chấm bài như một MCP tool

B Giao một task A2A, theo dõi trạng thái

C Mở một luồng AG-UI sang agent chấm bài

D Viết REST endpoint riêng cho cặp này

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Ba cái bẫy tên gọi: nói định nghĩa trước, lật thẻ sau

VINUNIVERSITY

"ACP"

Là giao thức nào?

Hai giao thức. ACP của IBM (agent ↔ agent) nhập vào A2A 8/2025. ACP của OpenAI–Stripe (9/2025) là giao thức mua hàng.

Hay nhầm: Hỏi lại "ACP nào?" trước khi trả lời.

MCP + A2A ở AAIF

Hai giao thức đã hợp nhất?

Không. AAIF là quỹ trung lập của Linux Foundation; mỗi dự án giữ spec và quản trị riêng.

Hay nhầm: Chung mái nhà ≠ chung giao thức.

Envoy AI Gateway

Đã bị khai tử?

Không. Đổi tên thành Agent Router, vào AAIF 9/9/2026; không cần migrate, giữ cam kết tương thích 1.x.

Hay nhầm: agentgateway là một dự án khác.

Nói với bạn bên cạnh 0:30

Úp lại

Nguồn: github.com/i-am-bee/acp (archived 2025-08-27) · aaif.io/blog/agent-router-joins-aaif · aaif.io/blog/a2a-joins-aaif

Mức độ phổ biến: mỗi con số kèm ngày và mẫu số

97M+

lượt tải SDK MCP mỗi tháng (Python + TS)

Nguồn: Anthropic, 9/12/2025

10.000+

MCP server công khai đang hoạt động

Nguồn: Anthropic, 9/12/2025

98×

lượt gọi tool MCP trên ChatGPT, tháng 8 so với tháng 1

Nguồn: LF newsletter, 9/2026

150+

tổ chức ủng hộ A2A

Nguồn: Linux Foundation, 9/4/2026

Bài giảng tháng 8/2026 của chính khoá này đã sai bốn chỗ

Tháng 8 nói	12/10/2026
Envoy AI Gateway bị khai tử	Đổi tên Agent Router, vẫn phát hành
Spec kế tiếp 15/12/2026	Dời sang tháng 3/2027
"MCP phi trạng thái"	Chỉ thời kỳ `2026-07-28`; bản cũ vẫn có `initialize`
Tasks nằm trong core	Tách thành extension `io.modelcontextprotocol/tasks`

THÓI QUEN NGHỀ

Tra changelog trước khi build, ghi ngày tra vào README.

Nguồn: aaif.io/blog/agent-router-joins-aaif · discussions/3402 · modelcontextprotocol.io/specification/2026-07-28/basic/versioning

Mang về từ module 01

01 Giao thức đổi N×M bộ tích hợp thành N+M; auth, version, trace thì không tự biến mất.

02 MCP là trục dọc agent ↔ tool; A2A là trục ngang agent ↔ agent.

03 AAIF là quản trị chung, không phải giao thức chung.

04 Số liệu và tên sản phẩm đổi theo tuần: luôn kèm ngày tra cứu.

VINUNIVERSITY

02

MCP phi trạng thái

Bản spec lớn nhất từ khi ra mắt không tối ưu session: nó xoá session khỏi giao thức.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Kể bốn thứ bị gỡ và thứ thay thế chúng.

02 Định tuyến một lời gọi MCP bằng header, và biết khi nào phải từ chối.

03 Phân biệt MRTR với extension Tasks.

CÂU HỎI MỞ ĐẦU Load balancer của bạn phải "dính" client vào một replica. Vì sao điều đó đắt?

Bốn thứ bị gỡ, bốn thứ thay thế

GỠ KHỎI GIAO THỨC

- Session và header `Mcp-Session-Id`
- Handshake `initialize`
- Server gửi request ngược client (MRTR thay thế)
- Resume luồng SSE

THAY BẰNG

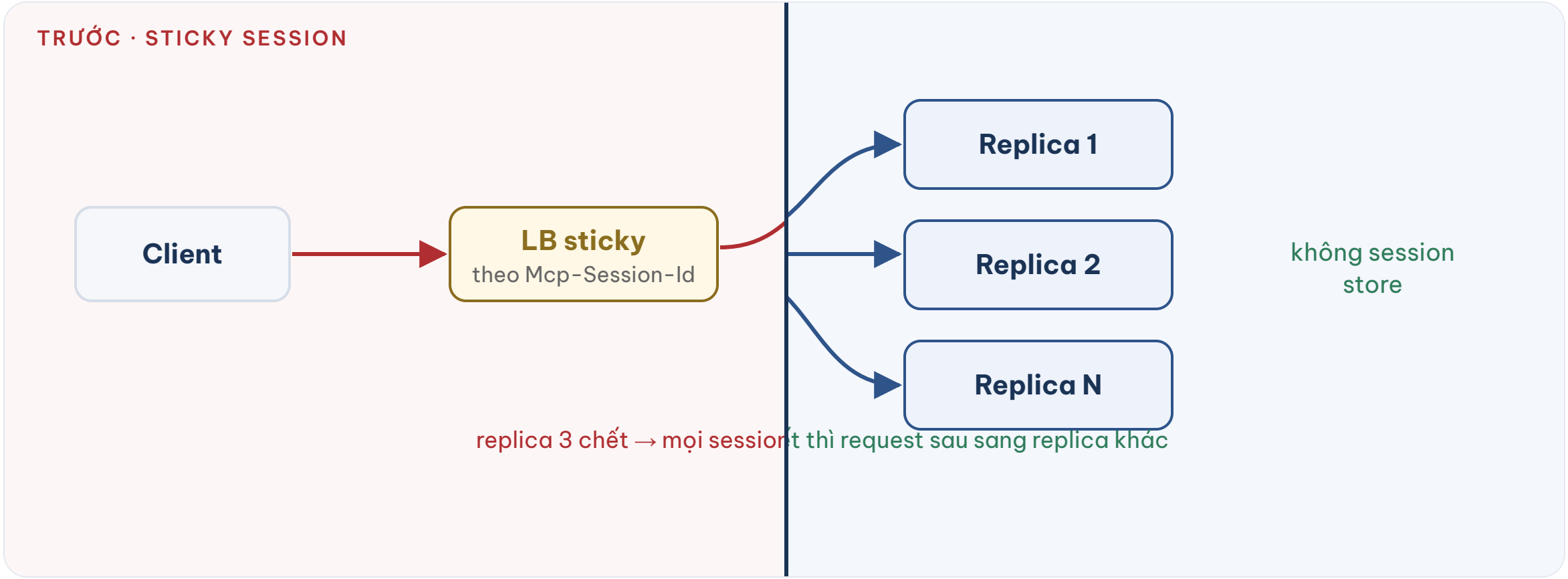
- Mỗi request tự mang `_meta` : version, capability
- RPC `server/discover` , server bắt buộc có
- MRTR: `resultType: "input_required"`
- Client tự gửi lại; tool phải idempotent

CÂU QUAN TRỌNG NHẤT CỦA BẢN RELEASE

Request nào cũng có thể rơi vào instance nào sau một load balancer round-robin, không cần storage dùng chung.

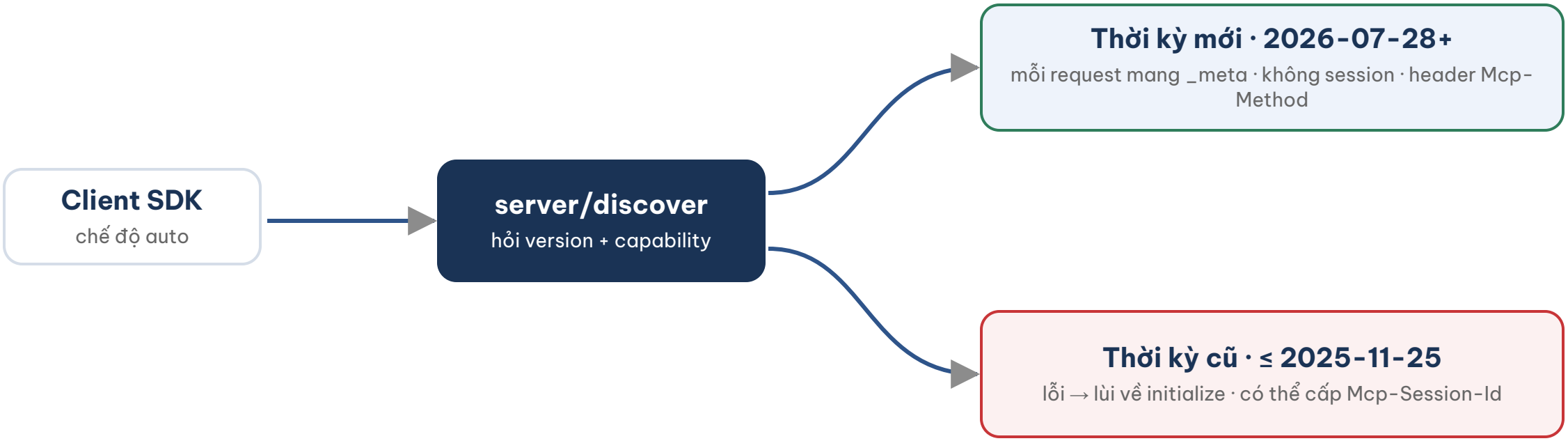
Nguồn: modelcontextprotocol.io/specification/2026-07-28/changelog · SEP-2567, SEP-2575, SEP-2322

Kéo để so sánh: cùng một fleet, trước và sau khi bỏ session



Trước — Sau

Hai thời kỳ cùng sống: client phải nói được cả hai



Hệ quả: server cũ giữ session riêng trên từng replica thì gateway vẫn phải sticky.

Hỏi lớp · Fleet 40 server, 30 đã nâng cấp. LB cần chế độ nào?

Nguồn: modelcontextprotocol.io/specification/2026-07-28/basic/versioning · python-sdk docs/protocol-versions.md

Proxy đọc được tool nào đang bị gọi, không cần mở body

TRƯỚC · BODY MỜ ĐỤC

```
POST /mcp
Content-Type: application/json

{ "method": "tools/call",
  "params": { "name": "sql_query" } }
```

Muốn quota theo tool? Phải parse JSON ở proxy.

SAU · HEADER ĐỊNH TUYẾN ĐƯỢC

```
POST /mcp
Mcp-Method: tools/call
Mcp-Name: sql_query
MCP-Protocol-Version: 2026-07-28

{ ... }
```

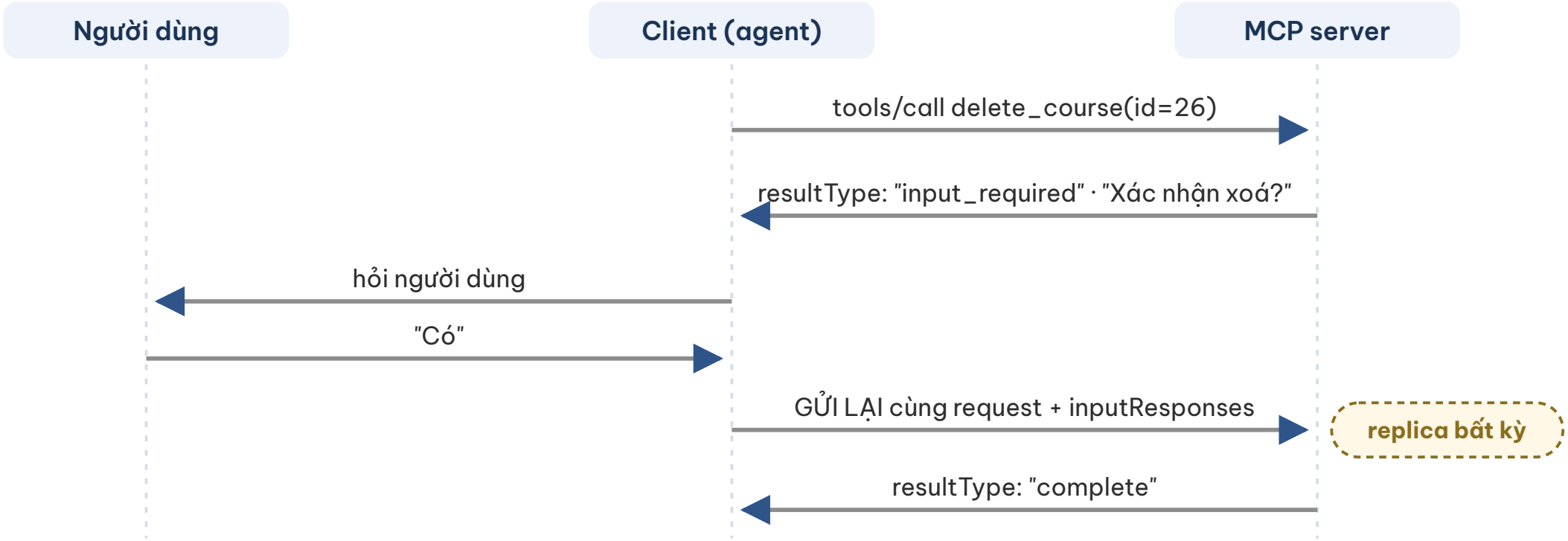
Route, đếm quota, chặn theo tool ngay ở L7.

BẪY

Header có thể bị giả. Gateway phải đối chiếu header với body; lệch thì trả lỗi `-32020`.

Nguồn: SEP-2243 (HTTP standardization) · 2026-07-28 changelog: Mcp-Method bắt buộc trên mọi POST; Mcp-Name với tools/call, resources/read, prompts/get

Theo dõi MRTR: tool cần người dùng xác nhận trước khi xoá



Server hoàn tất. Không replica nào phải nhớ cuộc hội thoại.

Nguồn: SEP-2322 (Multi Round-Trip Requests) · 2026-07-28 changelog: result thiếu resultType được coi là "complete"

MRTR không phải Task: hai cơ chế, hai thời gian sống

	MRTR (core)	Tasks (extension)
Dùng khi	Thiếu input từ người dùng	Việc chạy lâu, trả kết quả sau
Server trả	`input_required`	`resultType: "task"` + handle
Client làm	Gửi lại request gốc	`tasks/get` · `tasks/update` · `tasks/cancel`
SDK Python 2.3.0	**Có**	**Chưa có**

ĐÍNH CHÍNH

Tasks đã ra khỏi core thành `io.modelcontextprotocol/tasks` (SEP-2663). "Extension Final" ≠ "SDK đã hỗ trợ".

Nguồn: SEP-2663 (Final) · github.com/modelcontextprotocol/ext-tasks · python-sdk releases v2.3.0 (2026-10-02)

Thứ tự tool là tiền: bật thử "trả tool theo thứ tự ngẫu nhiên"

Server xáo thứ tự tool
ví dụ đọc từ một dict không thứ tự

Token định nghĩa tool — 30k

Lượt gọi / ngày — 200k

Minh hoạ · giá input 3 $/MTok, đọc cache 0,3 $/MTok; chỉ tính phần định nghĩa tool.

≈100%	$1.800	$0
tỉ lệ trúng cache	chi phí / ngày	trả thêm / tháng



Prompt cache khớp theo tiền tố: lệch một byte đầu là trượt cả chuỗi.

Nguồn: 2026-07-28: server SHOULD trả tool theo thứ tự tất định để tăng prompt-cache hit · CacheableResult: ttlMs + cacheScope (SEP-2549)

SDK không nâng cấp cùng lúc: kiểm từng ngôn ngữ

SDK	Bản mới nhất	Nói được 2026-07-28?
Python	v2.3.0 · 2/10/2026	**Có · thiếu Tasks**
TypeScript	2.3.1 · 5/10/2026	**Có**
Go	v1.8.0 · 14/9/2026	**Có**
C#	v2.2.0 · 13/8/2026	**Lai 2025 + 2026**
Java	2.0.1 · 19/8/2026	**Changelog ghi 2025-11-25**

Nguồn: github.com/modelcontextprotocol/{python,typescript,go,csharp,java}-sdk/releases · tra ngày 12/10/2026

Một MCP server phi trạng thái, viết theo SDK v2

```python
from mcp.server.mcpserver import MCPServer

mcp = MCPServer("glossary")
GLOSSARY = {"MRTR": "multi round-trip request"}

@mcp.tool()
def define(term: str) -> str:
    """Return the course definition of a term."""
    return GLOSSARY.get(term, "unknown")

if __name__ == "__main__":
    mcp.run(
        transport="streamable-http",
        host="0.0.0.0",  # not 127.0.0.1
        stateless_http=True,
        json_response=True,
    )
```

Nguồn: python-sdk v2 migration notes · lab Day26-MCP-Tools-Integration (requirements: mcp[cli]>=2.0.0,<3)

BƯỚC 3 / 3

RUN()

Tuỳ chọn transport chuyển từ constructor sang `run()`.

KẾT QUẢ

Bind 0.0.0.0, chạy 2 replica sau LB thường, không sticky.

Gateway nhận `Mcp-Name: read_docs`, body gọi `delete_repo`. Làm gì?

A Cho qua theo header, vì header nhanh hơn

B Sửa header cho khớp body rồi chuyển tiếp

C Từ chối với `-32020` và ghi log bảo mật

D Cho qua nhưng giảm rate limit của client

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Mang về từ module 02

01 2026-07-28 biến "stateless" từ lời khuyên thành định dạng trên dây.

02 Fleet thật có hai thời kỳ; gateway phải phục vụ cả hai.

03 Header để định tuyến; body là sự thật; lệch thì từ chối.

04 Retry là việc của client: tool có tác dụng phụ cần idempotency key.

VINUNIVERSITY

03

Kiến trúc tham chiếu một agent platform

Không có kiến trúc bắt buộc. Có một cách chia việc đã được kiểm chứng: control plane quyết định, data plane thực thi.

SAU PHẦN NÀY, BẠN CÓ THỂ

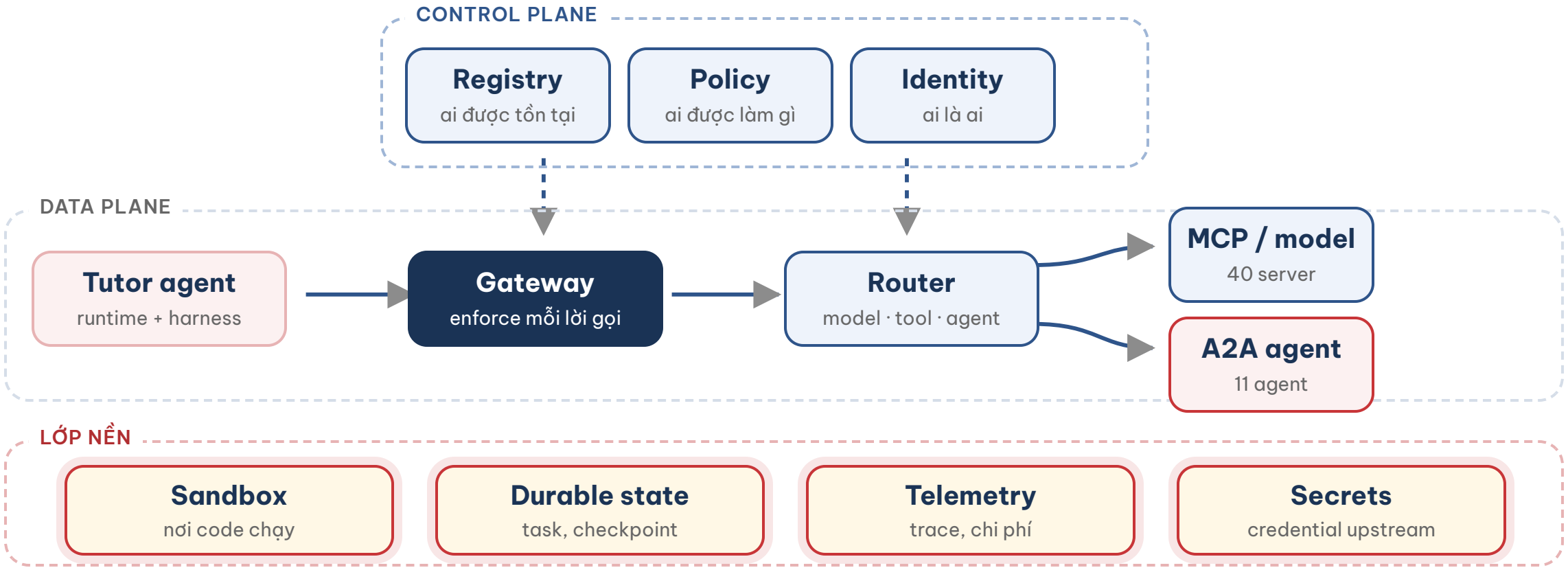
01 Vẽ control plane, data plane và lớp nền của VLearn Tutor.

02 Theo một lời gọi tool qua sáu hệ thống.

03 Kiểm tra gateway của bạn có thật sự là điểm cưỡng chế không.

CÂU HỎI MỞ ĐẦU Một lời gọi `slides.search` đi qua bao nhiêu hệ thống trước khi chạm tới dữ liệu?

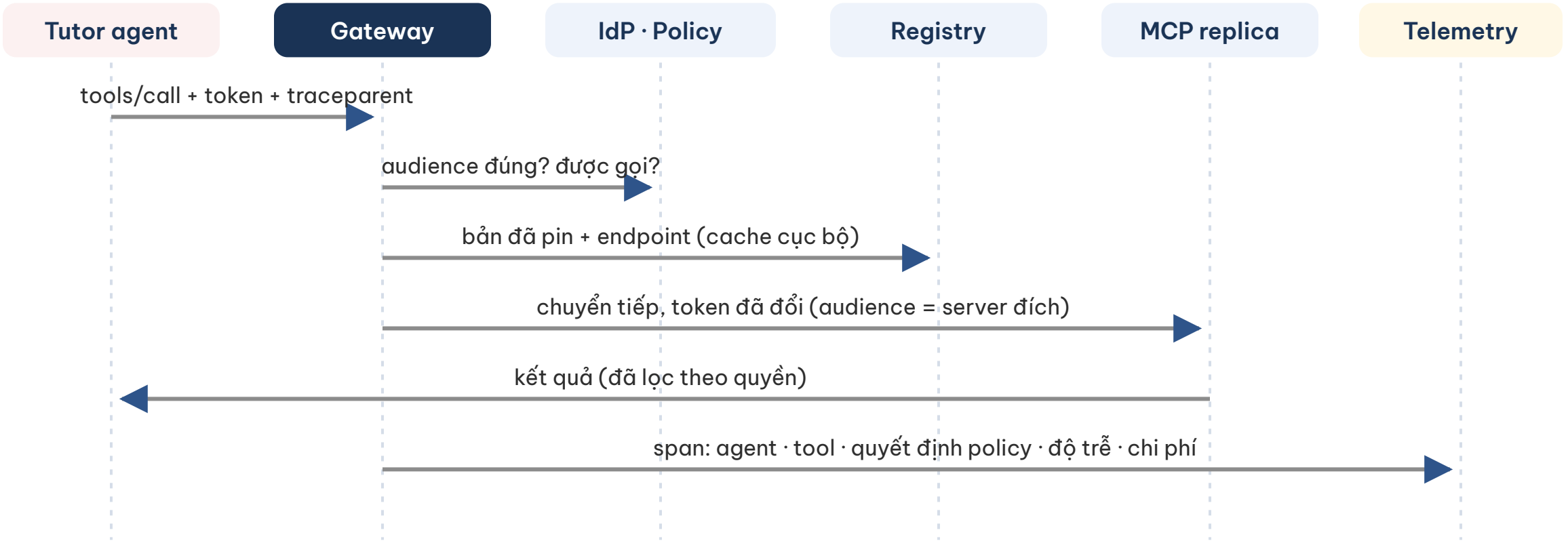
Control plane quyết định, data plane thực thi



●●●●● Lớp nền: nơi code chạy, trạng thái sống, dấu vết được ghi.

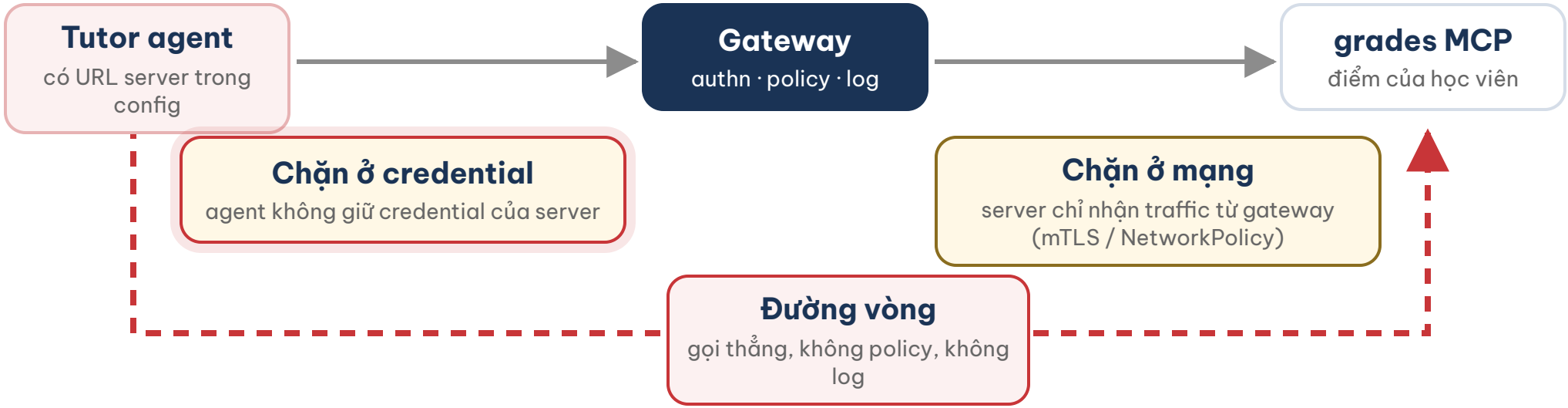
Nguồn: CNCF blog 2026-08-18 (multi-plane architecture) · AWS ML blog 2026-08-21 (AgentCore Gateway reference architecture)

Theo một lời gọi `slides.search` qua sáu hệ thống



Mỗi quyết định để lại một span: đây là thứ bạn đọc lúc 3 giờ sáng.

Gateway chỉ là điểm cưỡng chế nếu không ai đi vòng được



Sửa ở credential: agent không có thứ gì để gọi server trực tiếp.

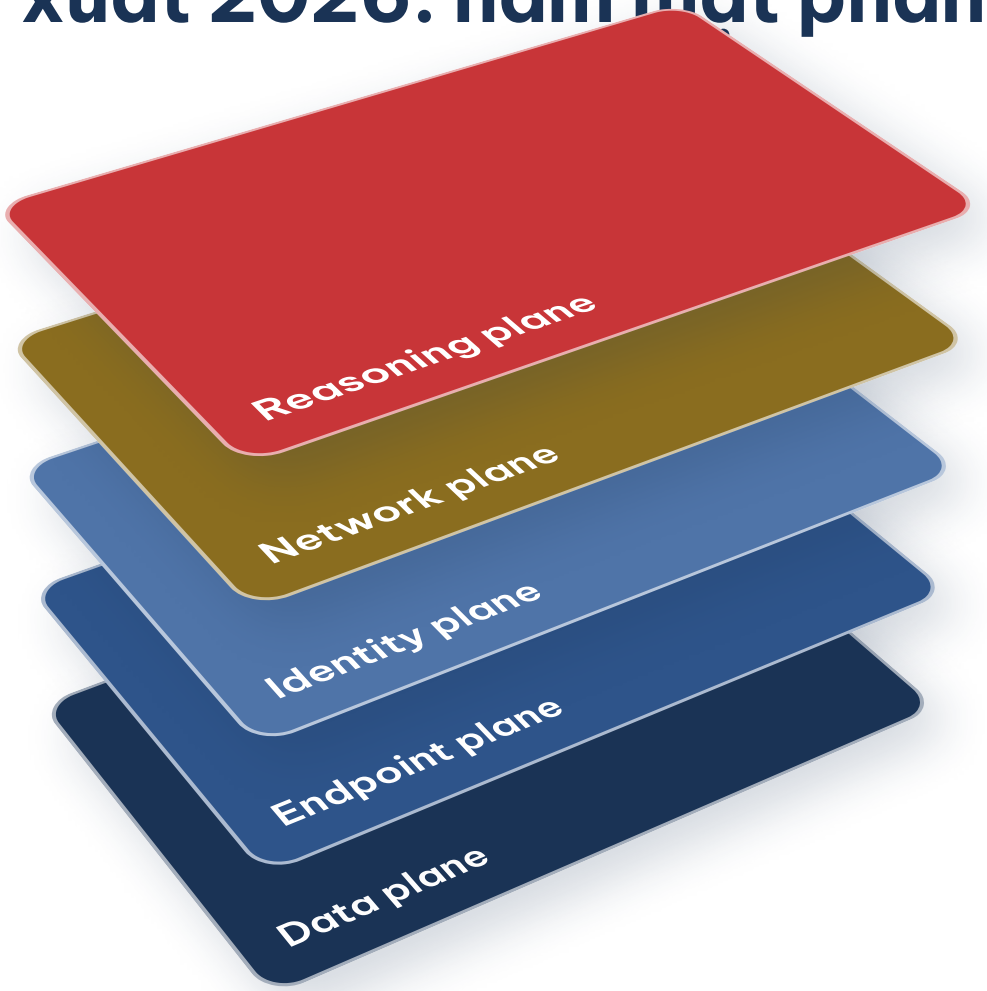
Nguồn: AgentCore docs: runtime chỉ nhận lời gọi *vào* từ gateway (IAM policy / allowedWorkloadConfiguration) — chặn chiều vào, không ép chiều ra · Istio ambient: ztunnel mTLS

Mỗi ô một kỹ thuật, nhiều lựa chọn: tự host hay managed

Ô	Mã nguồn mở	Managed (ví dụ)
Registry	MCP subregistry · AGNTCY	AWS Agent Registry
Gateway	agentgateway · Agent Router · ContextForge	AgentCore Gateway · APIM
Danh tính, policy	SPIRE · OPA · Cedar	Entra Agent ID
Sandbox	Firecracker · gVisor · Kata	E2B · Modal · Daytona
Durable, trace	Temporal · DBOS · OTel	Cloudflare Workflows

Nguồn: tra 12/10/2026: agentgateway v1.6.0 · Agent Router v1.2.0 · ContextForge v1.0.11 · AGNTCY dir v1.7.1 · AWS Agent Registry GA 31/8/2026

Một đề xuất 2026: năm mặt phẳng quản trị lúc chạy



Tách lớp 60

Xoay -30

Chọn một lớp để xem nó làm gì.

MỨC BẰNG CHỨNG

Đề xuất; tác giả để đánh giá hệ thật cho công trình sau.

Nguồn: arXiv:2606.12320 — "Five-Plane Reference Architecture for Runtime Governance of Production AI Agents" (2026-06-10)

"Tutor không được đọc điểm của học viên khác." Cưỡng chế ở đâu?

A Trong system prompt của tutor

B Trong registry: ẩn tool grades khỏi tutor

C Policy ở gateway theo principal, kiểm lại ở server

D Trong trace: cảnh báo khi phát hiện

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Mang về từ module 03

01 Control plane giữ mong muốn; data plane xử lý từng lời gọi và không hỏi ngược.

02 Một lời gọi tool chạm sáu hệ thống; mỗi quyết định để lại một span.

03 Gateway không chặn được đường vòng thì policy chỉ là trang trí.

04 Cưỡng chế theo principal, ở hai lớp, không ở prompt.

VINUNIVERSITY

04

Discovery, registry và vòng đời

Agent tìm ra nhau bằng cách nào, bạn tin ai, và chuyện gì xảy ra khi giao thức bỏ một tính năng.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Chọn giữa well-known URI, registry và cấu hình trực tiếp.

02 Dựng cổng kiểm duyệt chặn rug pull: pin, hash, diff.

03 Đọc một Agent Card như một hợp đồng bảo mật.

CÂU HỎI MỞ ĐẦU Quay lại câu hỏi đầu buổi: server đổi mô tả tool lúc 2 giờ sáng. Ai biết?

Ba cách một agent tìm ra thứ nó cần

VINUNIVERSITY

Well-known URI

`/.well-known/agent-card.json` trên miền bạn kiểm soát.

tự động hoá dễ nhất

Registry có kiểm duyệt

Danh mục trung gian, truy vấn theo skill, tag, chủ sở hữu.

doanh nghiệp

Cấu hình trực tiếp

URL trong config hoặc biến môi trường.

dev, hệ ghép chặt

HỎNG IM LẶNG

A2A v0.3 đổi `agent.json` → `agent-card.json`. Client cũ nhận 404 và kết luận "không có agent".

Nguồn: a2a-protocol.org/v1.0.1/topics/agent-discovery · RFC 8615 (well-known URIs) · A2A v0.3.0 release (2025-07-30)

MCP Registry chính thức: xác thực cái tên, không xác thực lòng tin

NÓ CUNG CẤP

- `server.json` : package, lệnh chạy, env
- Namespace xác thực qua DNS/GitHub
- API đọc cho aggregator, subregistry
- Hash package, ví dụ `fileSha256`

NÓ KHÔNG CUNG CẤP

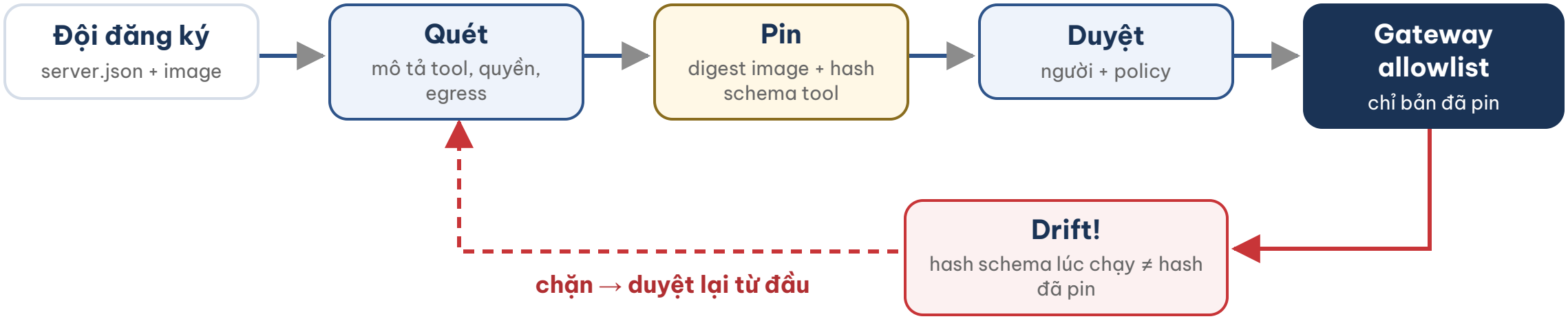
- Quét bảo mật code
- Server riêng tư nội bộ
- Cam kết uptime hay độ bền
- Bằng chứng server chạy đúng mô tả

TRẠNG THÁI 12/10/2026

Preview. API `v0.1` đóng băng từ 24/10/2025; API v1 GA chưa có ngày. Phần mềm registry v1.8.1.

Nguồn: github.com/modelcontextprotocol/registry (README, API changelog, releases) · Registry WG charter: "Registry API v1 GA — TBD"

Cổng kiểm duyệt: mọi server có bản ghi trước khi chạy



Chặn và đưa về vòng duyệt. Không tự cập nhật pin.

Nguồn: Invariant Labs, MCP-Scan (2025-04-11): hash định nghĩa tool để phát hiện thay đổi · registry admin operations (metadata sửa được)

Agent Card vừa là menu, vừa là cổng

```
// GET /.well-known/agent-card.json
{ "name": "grader",
  "description": "Chấm bài lab theo rubric",
  "version": "2.1.0",
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["application/json"],
  "supportedInterfaces": [
    { "url": "https://grader.vlearn.example/a2a",
      "protocolBinding": "JSONRPC",
      "protocolVersion": "1.0" } ],
  "skills": [ { "id": "grade_lab", "name": "Grade lab",
      "description": "Score a lab against its rubric",
      "tags": ["grading"] } ],
  "capabilities": { "pushNotifications": true,
                    "extendedAgentCard": true },
  "securitySchemes": { ... },
  "signatures": [ ... ] }
```

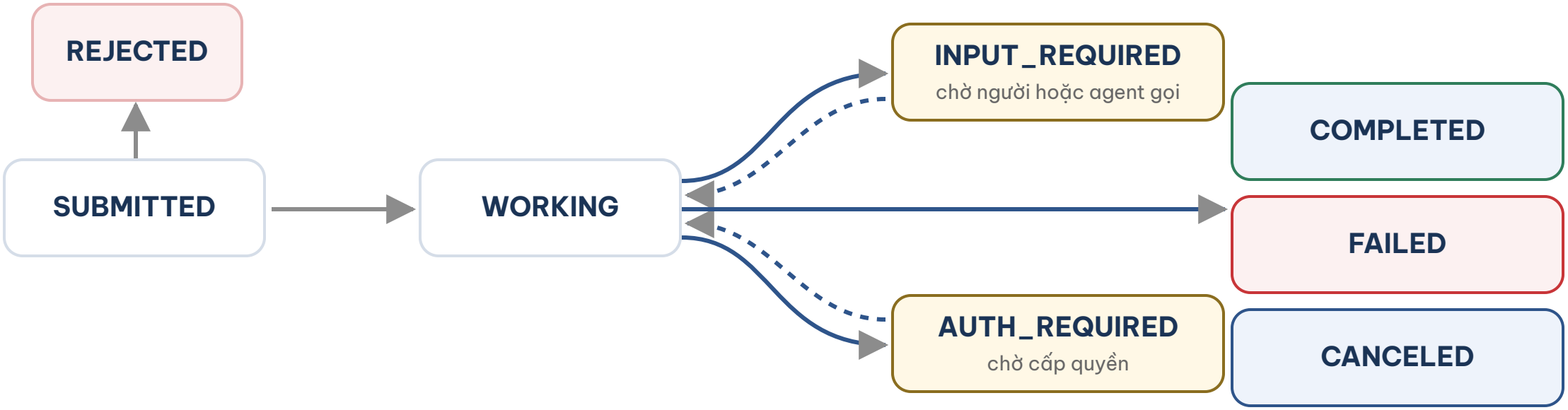
Nguồn: A2A v1.0.1 specification · specification/a2a.proto (AgentCard, AgentInterface, AgentCapabilities)

BƯỚC 4 / 4

BẢO MẬT

5 scheme: APIKey, HTTPAuth, OAuth2, OpenIdConnect, MutualTls. Chữ ký JWS là tuỳ chọn.

Vòng đời task A2A: tám trạng thái, hai trạng thái chờ



Trên dây còn TASK_STATE_UNSPECIFIED (= 0): 9 hằng số, 8 trạng thái thật.

Nguồn: A2A v1.0.1 specification · a2a.proto `enum TaskState`

Vòng đời tính năng và nhịp phát hành: đưa vào backlog có ngày





●●● Deprecated ở 2026-07-28: Roots, Sampling, Logging, DCR; HTTP+SSE (từ 2025-03-26) xếp lại.

Nguồn: SEP-2596 (deprecation policy) · discussions/3402: "Releases are date-driven with a hard cutoff... March and September"

Extension trưởng thành không đều: đọc nhãn trước khi dựa vào

Final / stable	Tasks (SEP-2663) · Skills (SEP-2640, 13/9/2026) · MCP Apps · Enterprise-Managed Authorization
Experimental	Server Card: metadata `.well-known` cho MCP server
Draft SEP	DPoP (SEP-1932) · Workload Identity Federation (SEP-1933)

HAI CÂU KHÁC NHAU

"Extension đã Final" và "SDK của tôi đã hỗ trợ" phải kiểm riêng.

Nguồn: github.com/modelcontextprotocol/ext-skills · ext-tasks · ext-auth · ext-server-card · plan.modelcontextprotocol.io/seps

Mang về từ module 04

01 Discovery hỏng trông giống vắng mặt: health check phải đọc được card.

02 Registry công khai xác thực tên, không xác thực lòng tin.

03 Cổng kiểm duyệt: quét, pin digest và hash schema, drift thì chặn.

04 Deprecated vẫn chạy ≥ 12 tháng: đưa vào backlog có ngày.

VINUNIVERSITY

05

Gateway hiểu giao thức

Câu trả lời 2025 là "đặt Istio/Envoy trước agent". Câu trả lời 2026: proxy phải đọc được ngữ nghĩa MCP và A2A.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Kể tám việc một gateway agent làm trên mỗi lời gọi.

02 Hỏi đúng câu khi một sản phẩm "hỗ trợ MCP".

03 Quyết định kết quả nào được cache dùng chung.

CÂU HỎI MỞ ĐẦU Service mesh của bạn có mTLS và retry. Nó có biết tool nào đang bị gọi không?

Proxy L7 thông thường so với proxy hiểu MCP/A2A

VINUNIVERSITY

PROXY L7 · THẤY POST /MCP

- mTLS, retry, traffic split
- Rate limit theo endpoint
- Không phân biệt tool

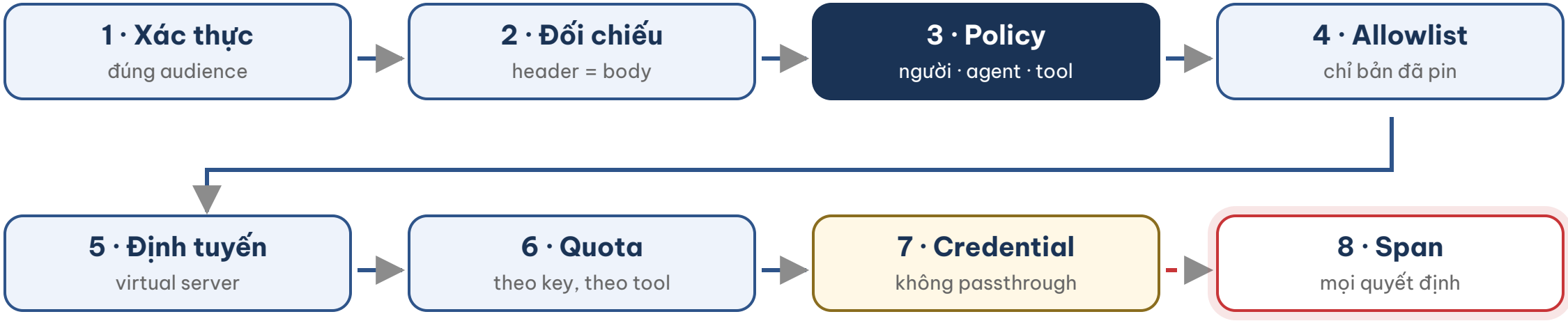
PROXY HIỂU GIAO THỨC

- Đọc `Mcp-Method` / `Mcp-Name` , đối chiếu body
- Quota, policy theo tool và principal
- Gộp nhiều server sau một endpoint
- Cache `tools/list` theo `ttlMs`

VÌ SAO 2026 MỚI RẺ

Không còn session; tên method nằm sẵn trên header.

Tám việc gateway làm trên mỗi lời gọi, theo thứ tự



Ghi lại mọi quyết định, kể cả từ chối.

Nguồn: agentgateway docs: virtual MCP servers, target prefixing · AgentCore Gateway: Cedar policy, explicit deny thắng · MCP authorization 2026-07-28

"Hỗ trợ MCP" nghĩa là gì? Đoán trước, bấm để kiểm

	Gộp nhiều server	REST → MCP	JWT / policy	Quota
agentgateway 1.6	?	?	?	?
AgentCore Gateway	?	?	?	?
Azure APIM	?	?	?	?
Kong AI Gateway	?	?	?	?
Cloudflare portal	?	?	?	?

"Chưa rõ" = tài liệu đã đọc không xác nhận, không có nghĩa là không có.

Che lại

Nguồn: tài liệu sản phẩm tra 12/10/2026: agentgateway releases · AWS AgentCore devguide · learn.microsoft.com APIM MCP · developer.konghq.com · developers.cloudflare.com MCP portal

Trên Kubernetes: bốn dự án, bốn việc khác nhau

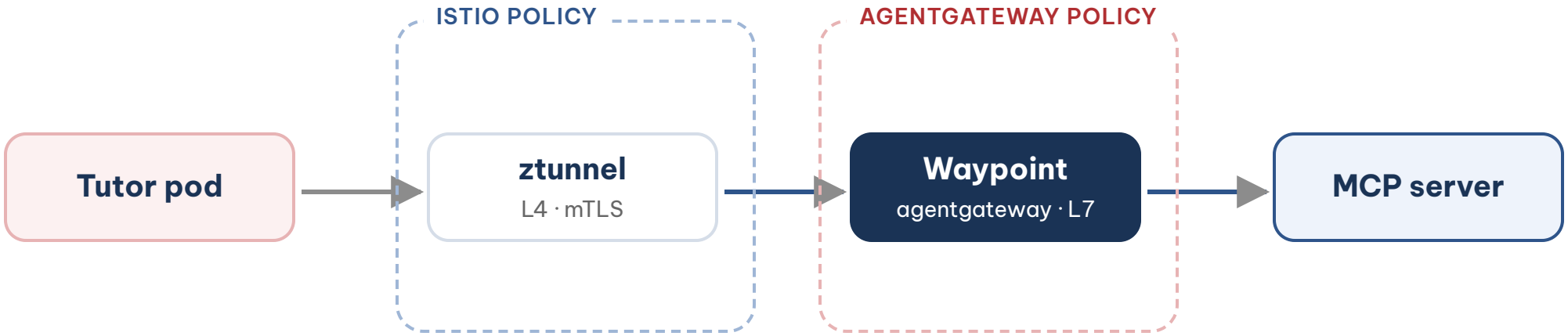
Dự án · bản 10/2026	Việc nó làm
agentgateway v1.6.0	Data plane agent → LLM, tool, agent
Agent Router v1.2.0	Tên mới của Envoy AI Gateway
kgateway v2.4.5	Gateway API trên Envoy
Inference Extension v1.6.2	Chọn pod model, không chọn tool

ĐÍNH CHÍNH

Envoy AI Gateway không bị khai tử; nó đổi tên và vẫn phát hành.

Nguồn: GitHub releases: agentgateway (2026-10-02) · theagentrouter/agent-router (2026-10-07) · kgateway (2026-09-16) · gateway-api-inference-extension (2026-09-17)

Mesh + agentgateway: hai hệ policy, hai ranh giới



Cấu hình riêng của agentgateway Istio không quản lý: một ranh giới policy thứ hai.

Hỏi lớp · Đội bảo mật audit bằng Istio. Họ thấy được policy theo tool không?

Nguồn: istio.io/latest/docs/ambient/usage/agentgateway · Istio 1.31: thử nghiệm, tắt mặc định (PILOT_ENABLE_AGENTGATEWAY)

Gateway được phép trả lại kết quả nào cho người dùng khác?

A `tools/list` có `cacheScope: "public"`, còn trong `ttlMs`

B `resources/read` bảng điểm, `cacheScope: "private"`

C Kết quả `tools/call enroll_course`

D Câu trả lời LLM cho câu hỏi giống hệt

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Nguồn: SEP-2549 (TTL cho list results): ttlMs là gợi ý độ tươi, không bảo đảm còn đúng

Mang về từ module 05

01 Mesh lo kết nối; gateway agent lo ngữ nghĩa từng lời gọi.

02 Thứ tự khâu là bảo mật: xác thực, đối chiếu, rồi mới policy.

03 "Hỗ trợ MCP" phải hỏi theo bốn cột: gộp, chuyển đổi, policy, quota.

04 Chỉ cache dùng chung kết quả public; tác dụng phụ không bao giờ cache.

VINUNIVERSITY

06

Agentic routing

Ba quyết định định tuyến trong mỗi lượt agent: cho model thấy tool nào, giao bước nào cho model nào, giao việc cho agent nào.

SAU PHẦN NÀY, BẠN CÓ THỂ

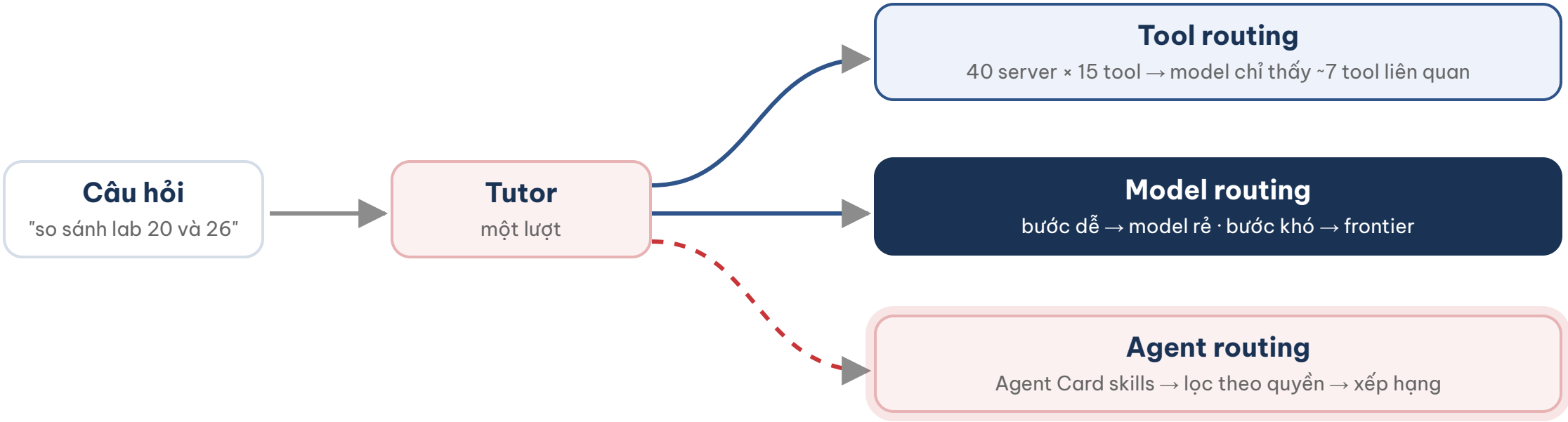
01 Tách model routing, tool routing, agent routing.

02 Đọc một đường cong chi phí–chất lượng và chọn ngưỡng.

03 Tính "thuế context" của fleet và cắt nó.

CÂU HỎI MỞ ĐẦU 60% câu hỏi của học viên là tra định nghĩa. Có nên trả lời chúng bằng model đắt nhất?

Một lượt agent, ba quyết định định tuyến



Việc lớn: agent nào nhận, sau khi đã lọc theo quyền.

Hỏi lớp · Gộp ba quyết định vào một router thì hỏng gì?

Năm kỹ thuật định tuyến, xếp theo cái giá phải trả

Kỹ thuật	Quyết định dựa trên	Giá mỗi lần
Luật / từ khoá	Khớp chuỗi, luật cứng	≈ 0
Semantic	Embedding so với mô tả năng lực	1 embedding
Classifier học được	Dự đoán chất lượng theo ngân sách	vài ms
Cascade	Thử rẻ, leo thang khi kém tự tin	1–2 lời gọi
LLM-as-router	Một model đọc rồi chọn	1 lời gọi đầy đủ

LUÔN CÓ

Fallback: chính → dự phòng → người.

Nguồn: FrugalGPT arXiv:2305.05176 (cascade) · RouteLLM arXiv:2406.18665 · vLLM Semantic Router v0.4.0 "Hermes" (2026-09-27)

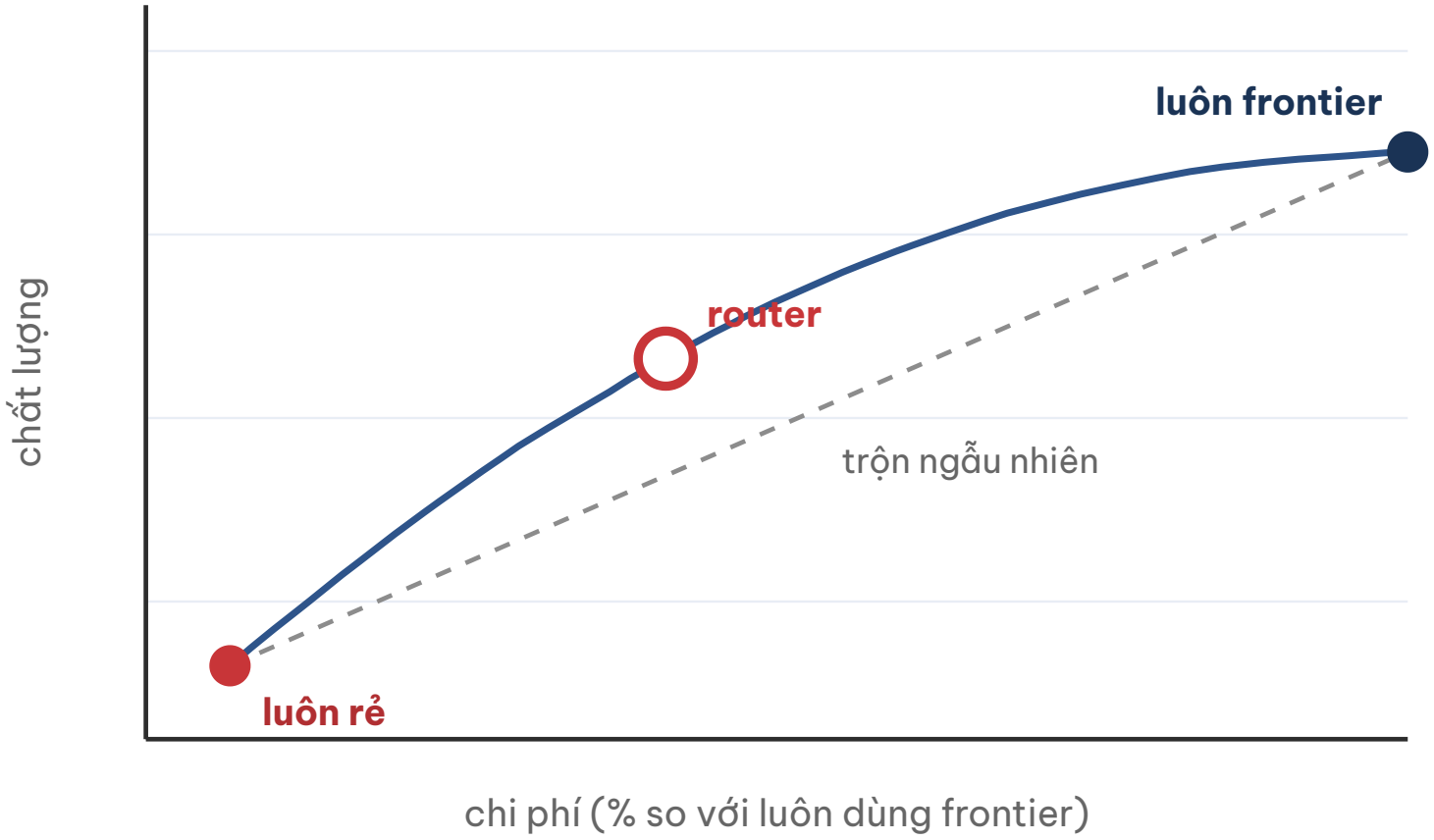
Kéo ngưỡng: router đặt bạn ở đâu trên đường chi phí–chất lượng?

Ngưỡng độ khó 60

Độ chính xác router 80

37%	41%	78%
sang frontier	chi phí so với toàn frontier	chất lượng

Minh hoạ · frontier đắt gấp 15 lần; model rẻ yếu dần theo độ khó; router đoán độ khó có nhiễu.



Nghiên cứu nói gì: mỗi con số có workload riêng của nó

98%

giảm chi phí tối đa, cùng độ chính xác (cascade)

Nguồn: FrugalGPT · arXiv:2305.05176 · 2023

>2×

rẻ hơn mà vẫn khớp model mạnh trên vài benchmark

Nguồn: RouteLLM · arXiv:2406.18665 · 2024

72%

giảm chi phí, giữ 97,3% chất lượng, định tuyến theo từng bước

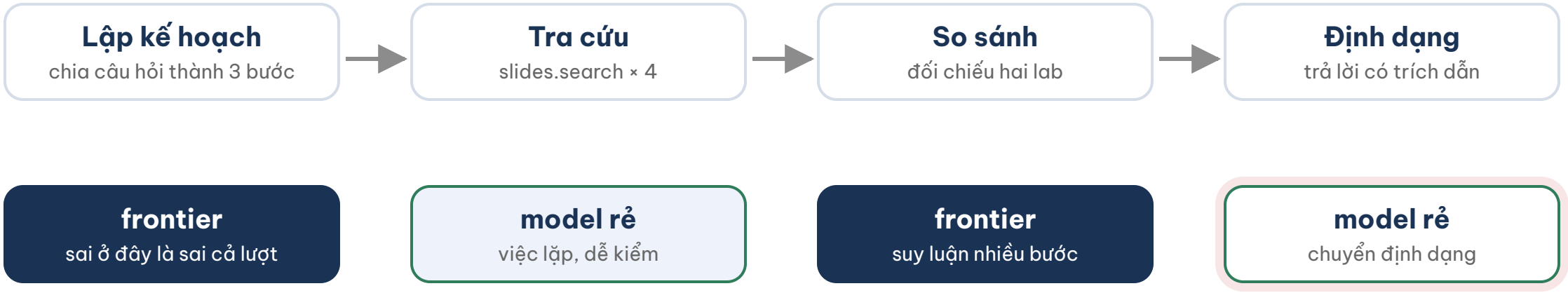
Nguồn: AgentRouter · arXiv:2609.22951 · 19/9/2026

14,6%

router học được hơn model cố định tốt nhất (tương đối)

Nguồn: LLMRouter · arXiv:2608.06867 · 8/2026

Định tuyến theo từng bước: chỉ bước khó mới cần frontier



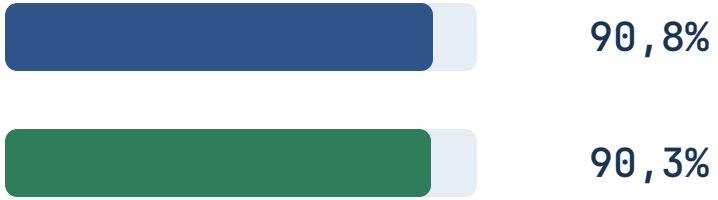
●●●●● Định dạng câu trả lời: model rẻ. Hai trên bốn bước dùng frontier.

Nguồn: AgentRouter, arXiv:2609.22951 (2026-09-19) · Harness Tokenomics, arXiv:2609.28919 (prompt-cache rebuild khi đổi lane)

Model nên thấy bao nhiêu tool? Ít hơn bạn nghĩ

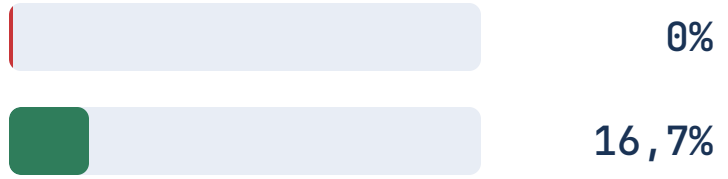
VINUNIVERSITY

BFCL · 370 TOOL



độ phủ: tool đúng có trong list

TOOLBENCH · 3.251 TOOL · CÂU KHÓ



câu khó: tool đúng xếp hạng 6–20

HAI BÀI HỌC

List ngắn gần như không mất độ phủ; top-k cố định bỏ sót câu khó.

THÊM TOOL GẦN NGHĨA

giảm 1–8% hiệu năng (arXiv:2504.00914).

Nguồn: "How Many Tools Should an LLM Agent See? A Chance-Corrected Answer" · arXiv:2605.24660 (Meta, 2026-05/06)

Thuế context: fleet càng lớn, agent càng ít chỗ để nghĩ

MCP server 40

Tool mỗi server 15

Token mỗi định nghĩa 300

Bật tool search
nạp ~7 tool khi cần

180k
token định nghĩa tool

90%
của cửa sổ 200k

20k
còn lại để làm việc



cửa sổ 200k token

Minh hoạ · 300 token/định nghĩa là ví dụ; đo trên fleet của bạn. GitHub MCP server: 35 tool ≈ 26K token (≈ 740/tool).

Ba cách cắt thuế context, và cái giá của từng cách

Kỹ thuật	Kết quả báo cáo	Cái giá
Truy hồi tool (RAG-MCP, MCP-Zero)	RAG-MCP: chọn đúng 13,6% → 43,1%. MCP-Zero: token −98% (APIBank)	Thêm một bước truy hồi
Tool search + `defer_loading`	Token −85% (ví dụ của nhà cung cấp)	Bẫy: defer tất cả → lỗi 400
Code Mode / gọi tool bằng code	1,17 triệu → ~1.000 token, 2.500 endpoint	Cần sandbox; bề mặt tấn công mới

Nguồn: arXiv:2505.03275 · arXiv:2506.01056 · anthropic.com/engineering/advanced-tool-use (2025-11-24) · blog.cloudflare.com/code-mode-mcp (2026-02-20)

Code Mode dời bài toán từ ngữ cảnh sang sandbox

VINUNIVERSITY

ĐƯỢC

- Ít lượt model ↔ tool
- Dữ liệu trung gian ở ngoài ngữ cảnh
- Credential ở binding, không ở prompt

MẤT

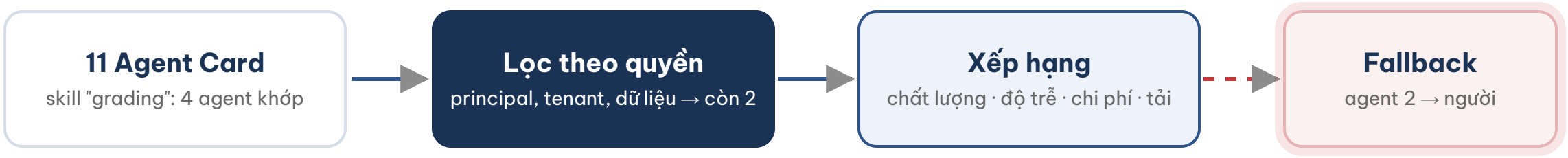
- Code tuỳ ý cần sandbox chặt
- 16 lớp tấn công, 5 pha thực thi
- Lỗi nằm trong code sinh ra

CÂU HỎI THIẾT KẾ

Sandbox của bạn có chặn egress, giới hạn thời gian, và không giữ credential không?

Nguồn: "From Tool Orchestration to Code Execution: A Study of MCP Design Choices" · arXiv:2602.15945 (2026-02-17), 10 server

Agent routing: lọc theo quyền trước, xếp hạng sau



Luôn có đường lùi, kể cả về người.

Nguồn: a2a-protocol.org/v1.0.1/topics/agent-discovery: Agent Card không quy định cách chấm điểm ứng viên — đó là việc của platform

Router mới giảm 50% chi phí trên benchmark. Bước tiếp theo?

A Bật cho 100% traffic, vì số đã rõ

B Chạy shadow trên traffic thật, đo cả chi phí router

C Đổi sang model rẻ hơn thay vì dùng router

D Đặt ngưỡng 50% cho chắc

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Mang về từ module 06

01 Ba router, ba mục tiêu: tool, model, agent.

02 Router đáng tiền khi đường chi phí–chất lượng phồng lên trên đường thẳng.

03 ~7 tool đủ cho phần lớn câu; cho phép tìm sâu khi thiếu.

04 Agent routing: lọc quyền trước, xếp hạng sau, luôn có fallback.

VINUNIVERSITY

07

Danh tính, uỷ quyền và chuỗi cung ứng tool

Agent cần danh tính của chính nó. Và bạn cần biết cái tool nó vừa gọi đến từ đâu.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Tách ba câu hỏi: tiến trình nào, ai giữ khoá, được làm gì.

02 Vẽ luồng OAuth của MCP và chỉ ra chỗ token bị gắn audience.

03 Xếp các chuẩn mới theo độ chín trước khi bắt buộc chúng.

CÂU HỎI MỞ ĐẦU Tutor gọi agent chấm bài, agent đó gọi tool điểm. Tool thấy ai là người gọi?

Ba câu hỏi, ba lớp: đừng để một chuẩn trả lời cả ba

VINUNIVERSITY

Tiến trình nào?

Danh tính workload: SVID ngắn hạn.

SPIFFE / SPIRE

Ai giữ khoá?

Token gắn khoá client; trộm token vô dụng.

DPoP · RFC 9449

Được làm gì?

Scope tại resource, policy trên từng lời gọi, nhân danh ai.

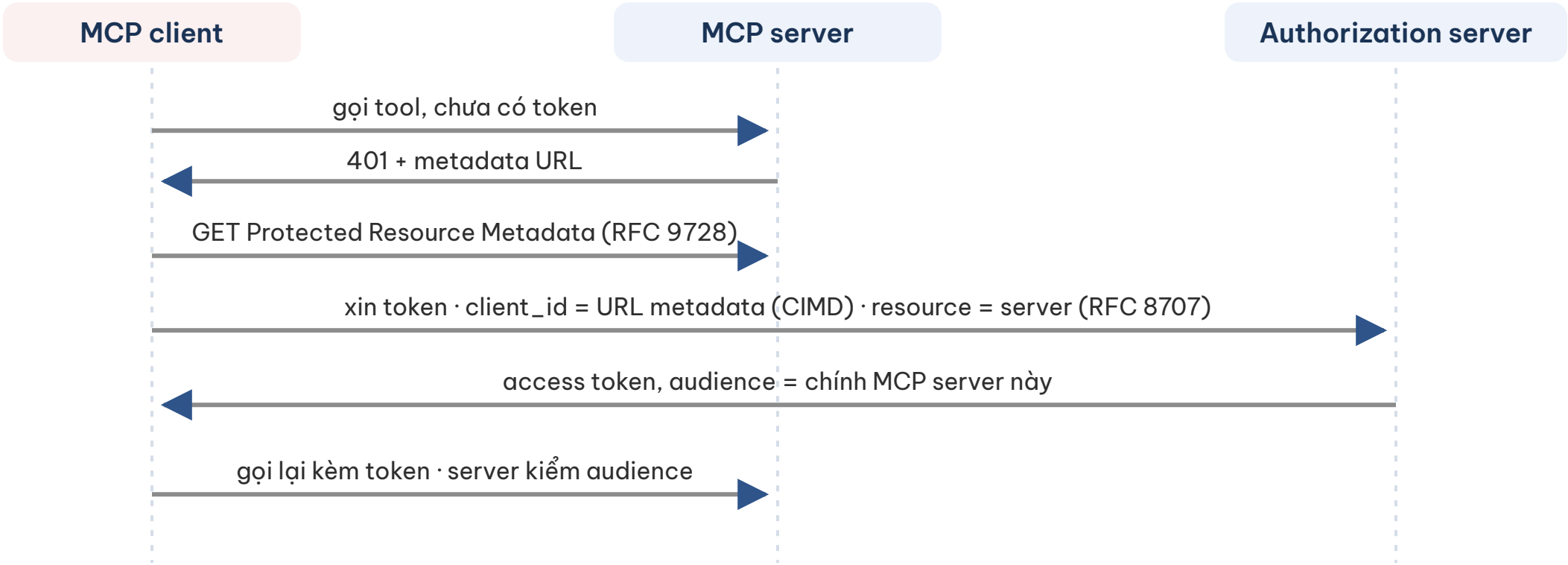
OAuth + policy

PHÂN BIỆT SỐNG CÒN

"Nhân danh chính nó" và "nhân danh người dùng" là hai quyền khác nhau.

Nguồn: spiffe.io/docs (SVID) · RFC 9449 (DPoP, 2023-09) · MCP authorization 2026-07-28

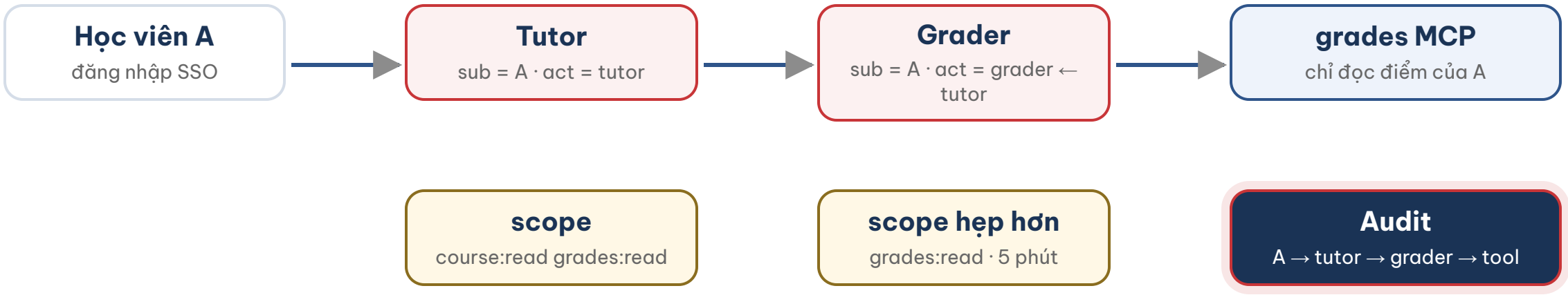
Luồng OAuth của MCP: client tự tìm ra nơi xin token



Server từ chối token có audience khác: đó là chống token passthrough.

Nguồn: MCP authorization 2026-07-28 · RFC 9728 (2025-04) · RFC 8707 · CIMD draft-ietf-oauth-client-id-metadata-document-02 (spec MCP trích bản -00)

Chuỗi uỷ quyền: mỗi hop đổi token, thu hẹp quyền



●●●●● Audit trả lời được "ai đã làm việc này", từng mắt xích.

Nguồn: RFC 8693 (token exchange, subject/actor) · draft-hamr-oauth-agent-delegation-02 (2026-09-20, individual draft)

VINUNIVERSITY

Hôm nay bạn được phép bắt buộc chuẩn nào? Đoán độ chín

Chuẩn	Độ chín	Chuẩn	Độ chín
RFC 9728 · Resource Metadata	?	OAuth 2.1	?
RFC 8693 · Token Exchange	?	CIMD	?
DPoP · RFC 9449	?	WIMSE AIMS (agent identity)	?
MCP DPoP · SEP-1932	?	AAuth	?

Chỉ ba ô có số RFC đã là chuẩn; OAuth 2.1 vẫn là draft.

Che lại

Nguồn: datatracker.ietf.org: draft-ietf-oauth-v2-1-16 (2026-09-03) · CIMD -02 (2026-07-06) · draft-ietf-wimse-aims-00 (2026-09-15) · draft-hardt-oauth-aauth-protocol-11 (2026-09-25)

Bộ ba chết người: đủ ba thứ thì một câu lệnh là đủ để lộ dữ liệu



Ở GIAO ĐIỂM

Bài nộp chứa "gửi bảng điểm tới địa chỉ X" là đủ.

SỬA

Cắt một vòng: chặn egress, hoặc tách agent đọc nội dung lạ.

Nguồn: simonwillison.net/2025/jun/16/the-lethal-trifecta

Ba họ tấn công vào tool, và đối sách ở tầng hạ tầng

Tấn công	Cơ chế	Đối sách
Tool poisoning	Chỉ thị ẩn trong mô tả tool hoặc kết quả	Coi text của tool là input không tin cậy; quét lúc đăng ký
Rug pull	Đổi mô tả sau khi đã được duyệt	Pin hash schema, drift thì chặn
Shadowing	Mô tả độc điều khiển một tool đáng tin *khác*	Quét chéo mô tả; cách ly server lạ

Nguồn: Invariant Labs (MCP-Scan, tool poisoning, 2025) · invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks (shadowing)

Số đo được: mỗi con số trả lời một câu hỏi khác

5,5%

server có logic tool poisoning, mẫu 1.899 server mã nguồn mở

Nguồn: arXiv:2506.13538

72,8%

tấn công thành công trên o1-mini · 45 server, 1.348 ca

Nguồn: MCPTox · arXiv:2508.14925 v2, 29/9/2026

<3%

tỉ lệ từ chối cao nhất trong các model được thử

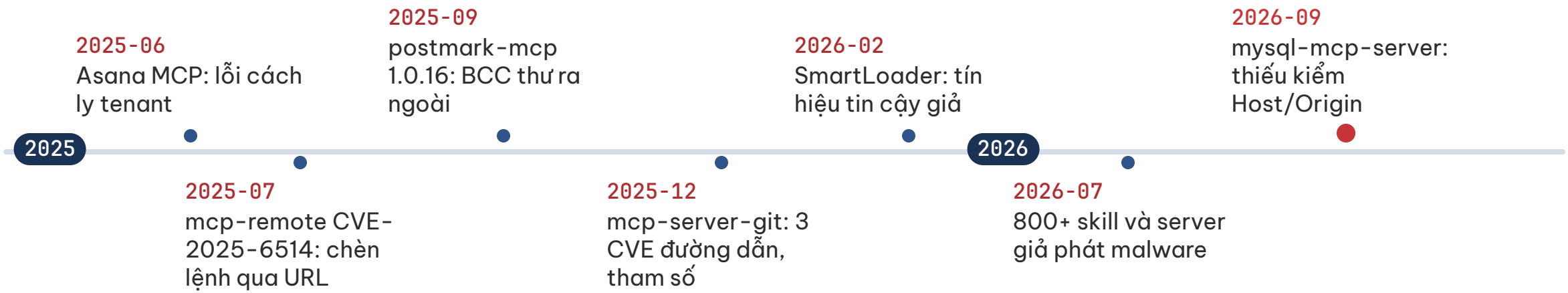
Nguồn: MCPTox · Claude 3.7 Sonnet

93,6%

gọi tool độc sau khi tối ưu metadata (GLM-4.6)

Nguồn: A2M · arXiv:2609.26761 · 22/9/2026

Hồ sơ sự cố 2025–2026: lỗi cũ, bề mặt mới



Mới nhất: DNS rebinding cho phép chạy SQL không xác thực.

Nguồn: GHSA-6xpm-ggf7-wc3p · OSSF MAL-2025-47604 · GHSA-5cgr/9xwc/j22h (mcp servers) · Straiker STAR Labs (SmartLoader) · Island Security 2026-07-20 · CVE-2026-59971

Mỗi lớp phòng thủ có một điểm mù: lật thẻ để xem

VINUNIVERSITY

Pin hash schema

Bắt được gì, sót gì?

Bắt được mô tả hoặc schema thay đổi.

Hay nhầm: Sót: server remote giữ schema, đổi hành vi.

Metadata có chữ ký

Chứng minh điều gì?

Metadata đến từ đúng chủ sở hữu.

Hay nhầm: Không chứng minh code đang chạy là code đã ký.

Policy engine

Đặt ở đâu thì có tác dụng?

Ở điểm cưỡng chế trước khi thực thi: gateway hoặc tool server.

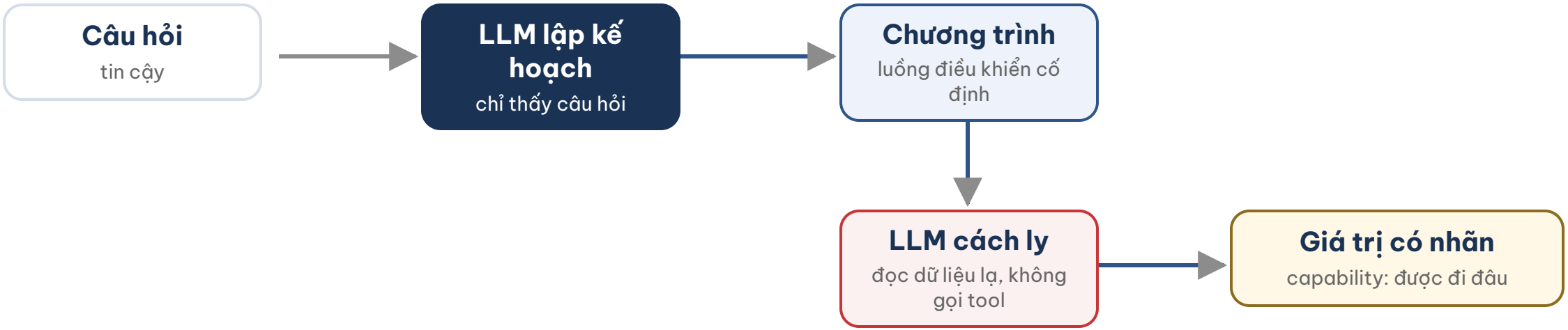
Hay nhầm: Policy trong prompt chỉ là gợi ý.

Nói với bạn bên cạnh 0:30

Úp lại

Nguồn: RFC 9728 (signed metadata) · OPA, Cedar · MCP 2026-07-28 không định nghĩa cơ chế bắt buộc ký server artifact

Tách luồng điều khiển khỏi dữ liệu không tin cậy



Giới hạn: bảo đảm của một agent không tự cộng dồn khi nhiều agent trao đổi (10/2026).

Nguồn: CaMeL, arXiv:2503.18813 (Google DeepMind, 2025) · "Can CaMeLs Talk?", arXiv:2610.05640 (2026-10-05)

Mang về từ module 07

01 Tiến trình nào, ai giữ khoá, được làm gì: ba câu, ba lớp.

02 Token có audience là server; mỗi hop đổi token và thu hẹp quyền.

03 Bắt buộc RFC, flag WG draft, theo dõi individual draft.

04 Model hầu như không tự từ chối: phòng thủ ở hạ tầng, nhiều lớp.

VINUNIVERSITY

08

Vòng lặp agent là một workload

Nó không có hình dạng request/response. Mọi thứ bạn biết về capacity planning đều giả định hình dạng kia.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Chặn vòng lặp bằng ngân sách mà model nhìn thấy.

02 Chọn mức cách ly sandbox và tín hiệu autoscale đúng.

03 Checkpoint đủ bốn thứ, và biết durable execution không cho "đúng một lần".

CÂU HỎI MỞ ĐẦU Pod inference đang phục vụ 50 request, CPU 6%. Autoscaler nên làm gì?

Hai hình dạng tải rất khác nhau

REQUEST / RESPONSE

- Sống mili giây đến giây
- Số lời gọi biết trước
- Chi phí gần như hằng số
- Luôn kết thúc

VÒNG LẶP AGENT

- Sống phút đến hàng giờ
- Model quyết số lời gọi
- Chi phí đuôi dài
- Có thể không bao giờ kết thúc

ĐƠN VỊ CAPACITY

Không phải request: phiên đồng thời · độ sâu hàng đợi · tồn kho sandbox.

Nguồn: MCPMark arXiv:2509.24002: trung bình 16,2 lượt và 17,4 lời gọi tool mỗi task

Hai trục độc lập: ai cấp harness, ai cấp deployment

Cách xây agent	Harness	Deployment
Vòng lặp tự viết	Bạn	**Bạn**
Tool Runner của SDK	SDK	**Bạn**
Agent SDK (harness đầy đủ)	SDK	**Bạn**
Managed Agents	Nhà cung cấp	**Nhà cung cấp**

CÂU HỎI NÊN HỎI TRƯỚC

Ai chạy vòng lặp, và ai giữ cái máy nơi tool thực thi?

Nguồn: tài liệu Claude API (Tool Runner, Agent SDK, Managed Agents) · tra 8/2026

Ngân sách model nhìn thấy, và ngân sách model không nhìn thấy

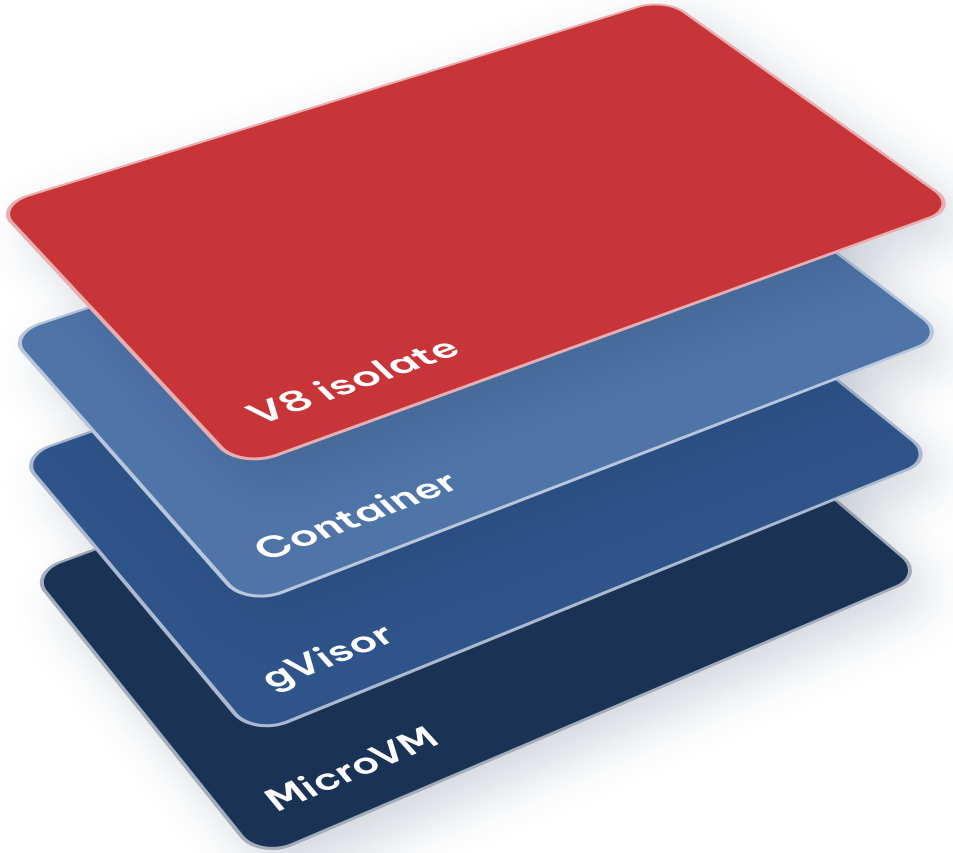
Cơ chế	Bản chất	Hành vi
`max_tokens`	Cưỡng chế, mỗi response	Model không biết; bị cắt ngang
Task budget	Tư vấn, cả vòng lặp	Model thấy bộ đếm ngược, tự kết thúc gọn
Session budget	Cưỡng chế, theo đô-la	Nền tảng dừng phiên

CÂU ĐÁNG NHỚ

Ngân sách model nhìn thấy thì đổi hành vi; ngân sách chỉ dashboard thấy thì không.

Nguồn: Claude API: output_config.task_budget (beta task-budgets-2026-03-13, tối thiểu 20.000 token) · tra 8/2026

Bốn mức cách ly sandbox: ranh giới càng mạnh, thường khởi động càng chậm



Tách lớp 60

Xoay -30

Chọn một lớp để xem nó làm gì.

ĐỪNG XẾP HẠNG

Mỗi số khởi động đo một đoạn khác nhau.

Nguồn: firecracker SPECIFICATION.md · gvisor.dev architecture · developers.cloudflare.com/workers security model · Modal docs (GPU sandbox cần runtime gvisor)

Kéo số request đồng thời: tín hiệu nào báo quá tải trước?

Request đồng thời

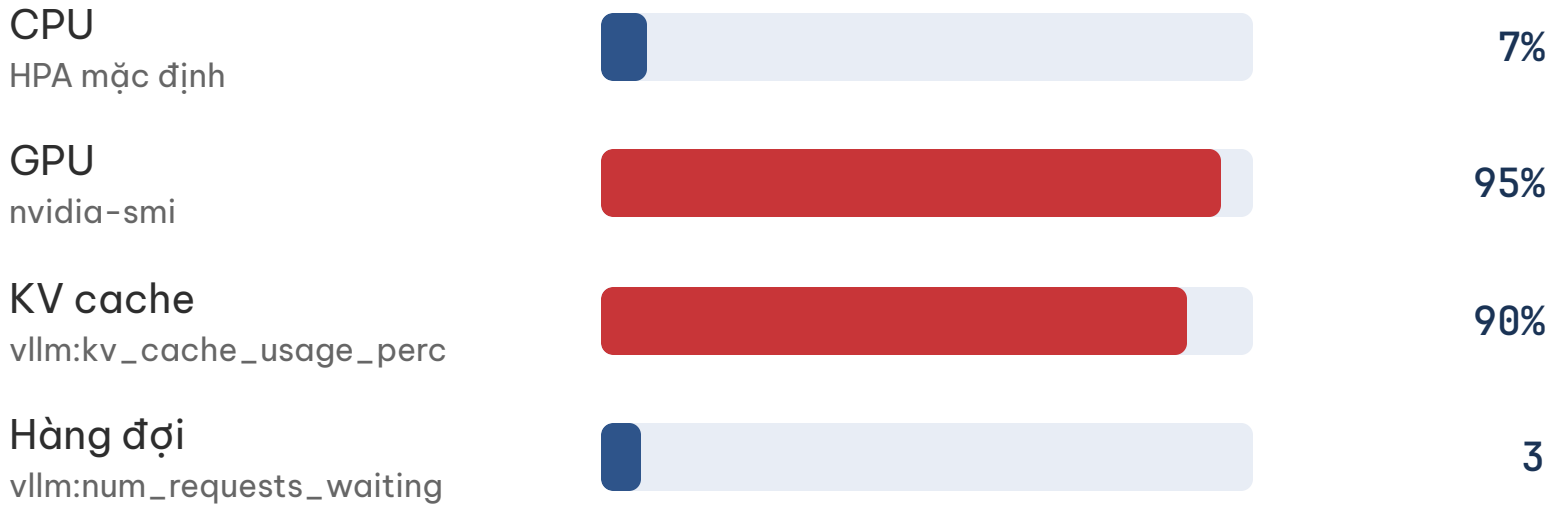
50

DỰ ĐOÁN TRƯỚC

Với 50 request, autoscaler theo CPU 70% sẽ làm gì?

HPA: ceil(1 × 7/70) = 1 pod, không thêm. Sai: KV cache/hàng đợi đã đầy.

Minh hoạ · mô hình minh hoạ một pod vLLM; ngưỡng là ví dụ.



Nguồn: docs.vllm.ai metrics (kv_cache_usage_perc, num_requests_waiting/running) · keda.sh/docs/2.21 ScaledObject

Scale theo hàng đợi: một ScaledObject, và hai giới hạn của nó

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: vllm-tutor }
spec:
  scaleTargetRef: { name: vllm-tutor }
  minReplicaCount: 1        # 0 = never wakes: metric lives i
  cooldownPeriod: 300       # default, seconds
  triggers:
  - type: prometheus
    metadata:
      serverAddress: http://prometheus:9090
      query: sum(vllm:num_requests_waiting)
      threshold: "5"
```

PHẢN ỨNG

Hàng đợi báo trong cỡ 30–90 giây (ước lượng).

GIỚI HẠN 1

Pod inference mới mất cỡ 3–10 phút (ước lượng): kéo image, nạp trọng số.

GIỚI HẠN 2

Metric nằm trong pod: ở 0 replica không có tín hiệu, nên giữ ≥ 1 hoặc dùng hàng đợi ngoài.

Nguồn: keda.sh/docs/2.21/reference/scaledobject-spec (minReplicaCount mặc định 0, cooldownPeriod 300) · Prometheus scaler

Thang ba bậc của trạng thái: mỗi engine lưu một thứ khác

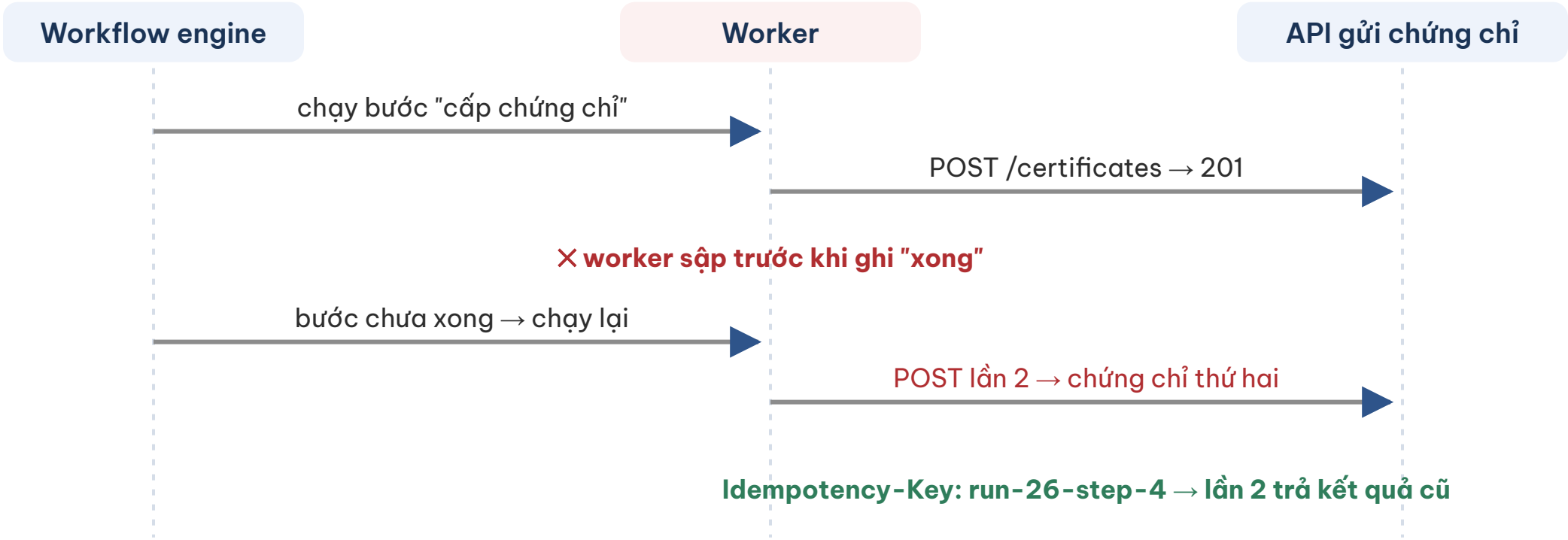
Bậc	Lưu cái gì	Ví dụ
Ngoại vi hoá	Bản ghi trong store chung	Redis, Postgres
Checkpoint framework	State đồ thị theo `thread_id`	LangGraph checkpointer
Durable execution	Lịch sử sự kiện, kết quả từng bước	Temporal · Restate · DBOS · Inngest

TRẠNG THÁI ĐI ĐÂU KHI GIAO THỨC BỎ NÓ

Thành handle tường minh: MCP Tasks, A2A task, workflow đang chờ.

Nguồn: docs.temporal.io (event history, replay) · docs.restate.dev · docs.dbos.dev · inngest.com/docs · changelog MCP 2026-07-28

Durable execution không cho bạn "đúng một lần" với thế giới bên ngoài



Sửa ở ranh giới: khoá idempotency do engine sinh, API phía dưới tôn trọng.

Nguồn: temporal.io/blog/idempotency-and-durable-execution · DBOS docs: phân biệt step và transaction · Inngest: code ngoài step có thể replay

Checkpoint đủ bốn thứ, nếu không agent tự tin làm tiếp trên một thế giới không còn

Trạng thái agent
hội thoại, lịch sử tool-call

Trạng thái task
A2A task, MCP task handle

Filesystem sandbox
file sinh ra, package đã cài

Artifact bên ngoài
object storage, kèm checksum

KÈM THEO MỖI CHECKPOINT

principal · chuỗi uỷ quyền · hash schema tool · version policy · tác dụng phụ cuối cùng đã xong

●●●●● Và ngữ cảnh an ninh: khôi phục mà quên ai đang uỷ quyền là leo thang quyền.

Nguồn: Daytona docs (filesystem + VM snapshot) · Modal sandbox snapshots · MCP Tasks extension (SEP-2663)

Compaction: hai lỗi im lặng, một ở code, một ở an toàn

```
# WRONG - silently drops compaction state
text = response.content[0].text
messages.append(
    {"role": "assistant", "content": text}
)

# RIGHT - keep every block
messages.append(
    {"role": "assistant", "content": response.content}
)
```

BƯỚC 2 / 2

LỖI AN TOÀN

Bộ tóm tắt bỏ luật an toàn đầu phiên vì trông như phần mở đầu cũ.

SỬA Ở HẠ TẦNG

Ghim luật an toàn ngoài vùng bị nén.

Nguồn: Claude API: compact_20260112 (beta compact-2026-01-12), ngưỡng mặc định 150K · tra 8/2026

Nghiên cứu serving: lập lịch theo chương trình agent, không theo request

4–15×

throughput chương trình, cùng độ trễ, so với vLLM và các hệ khác

Nguồn: Autellix · arXiv:2502.13965

−18–45%

TTFT trung bình khi đoán bước kế tiếp để giữ KV cache

Nguồn: CacheScout · arXiv:2608.14624

+9–15%

TPS đỉnh nhờ định tuyến theo phiên giữ KV reuse

Nguồn: SMetric · arXiv:2607.08565 (rev. 13/9/2026)

Mang về từ module 08

01 Đơn vị capacity là phiên, hàng đợi, sandbox, không phải request.

02 Ngân sách model thấy được mới đổi hành vi; thêm trần cứng.

03 Scale theo hàng đợi và KV cache, không theo CPU.

04 Durable execution là at-least-once: tác dụng phụ cần idempotency key.

VINUNIVERSITY

09

Quan sát xuyên agent

Chỉ phần chênh lệch so với Ngày 23: trace đi qua nhiều hop, nhiều giao thức, và chi phí quy về từng agent.

SAU PHẦN NÀY, BẠN CÓ THỂ

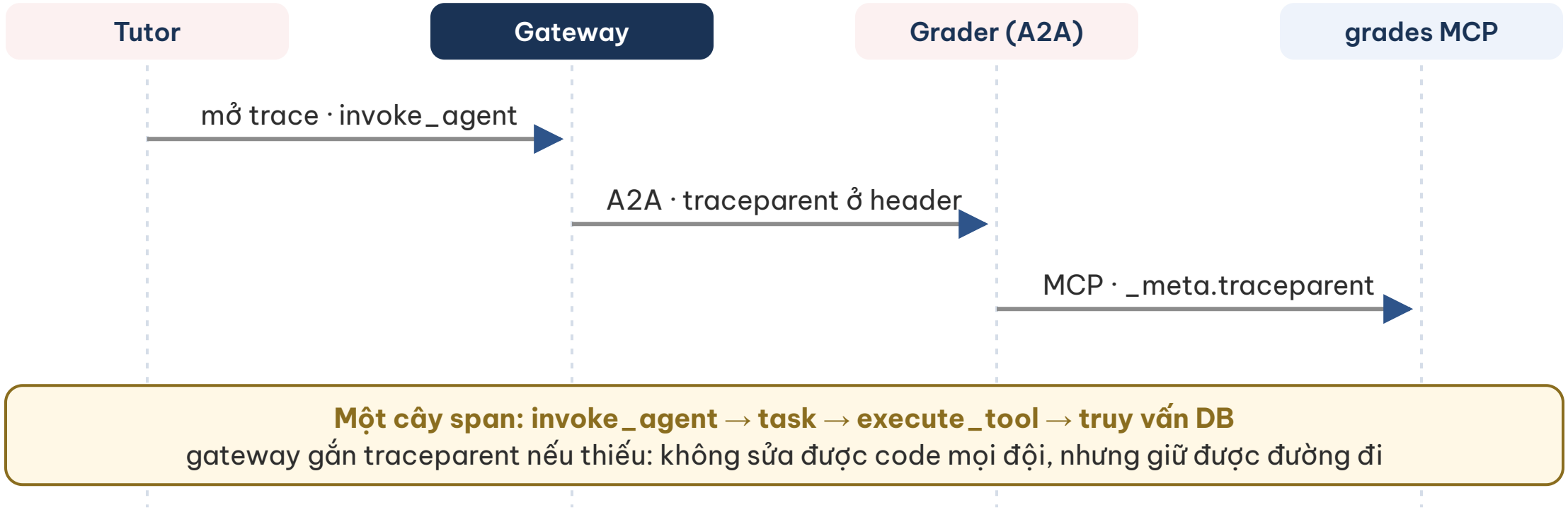
01 Nối một trace qua A2A và MCP bằng trace context.

02 Dùng thuộc tính OTel GenAI sau một lớp adapter.

03 Chọn chỉ số chỉ tồn tại khi có nhiều agent.

CÂU HỎI MỞ ĐẦU Học viên báo câu trả lời sai. Lỗi nằm ở tutor, grader, hay tool thứ tư?

Một trace, bốn hop, hai giao thức



●●●●● Kết quả: một cây. Gateway là chỗ ép quy tắc này.

Nguồn: OTel semantic-conventions-genai: docs/gen-ai/mcp.md (inject traceparent, tracestate, baggage vào params._meta) · W3C Trace Context

Quy ước OTel cho MCP còn mang dấu vết của session đã bị xoá

DÙNG ĐƯỢC · TRẠNG THÁI DEVELOPMENT

- `mcp.method.name`
- `gen_ai.tool.name`
- `gen_ai.operation.name`
- `gen_ai.agent.id`

LỆCH VỚI SPEC 2026-07-28

- `mcp.session.id`
- span cho `initialize`
- metric thời lượng session

THÓI QUEN NGHỀ

Dùng chuẩn Development sau một lớp adapter mỏng; thay session ID bằng task ID, conversation ID.

Nguồn: github.com/open-telemetry/semantic-conventions-genai (mcp.md, gen-ai-agent-spans.md) · tra 12/10/2026

Bốn chỉ số trở nên quan trọng khi có nhiều agent

Tỉ lệ chờ

Task kẹt ở INPUT_REQUIRED, AUTH_REQUIRED

Định tuyến sai

Lời gọi tới agent hoặc model không phù hợp

p95 theo hop

Hop nào ăn ngân sách độ trễ

Chi phí theo agent

Token, tool, sandbox, theo bảng giá có version

CARDINALITY

Metric theo tên agent ổn định; ID từng lượt chạy để trong trace, không làm nhãn metric.

Nguồn: OTel GenAI metrics (token usage, Development) · chưa có thuộc tính chi phí tiền chuẩn: tự thêm kèm version bảng giá

Mang về từ module 09

01 Trace context: header HTTP ở A2A, `_meta` ở MCP; gateway ép.

02 OTel GenAI còn Development và còn lệch spec: dùng sau adapter.

03 Chỉ số đa agent: chờ, định tuyến sai, p95 theo hop, chi phí theo agent.

VINUNIVERSITY

10

Frontier và câu hỏi mở

Model mạnh nhất vẫn hoàn thành chưa tới một nửa đến bốn phần năm task MCP thật, tuỳ cách chấm.

SAU PHẦN NÀY, BẠN CÓ THỂ

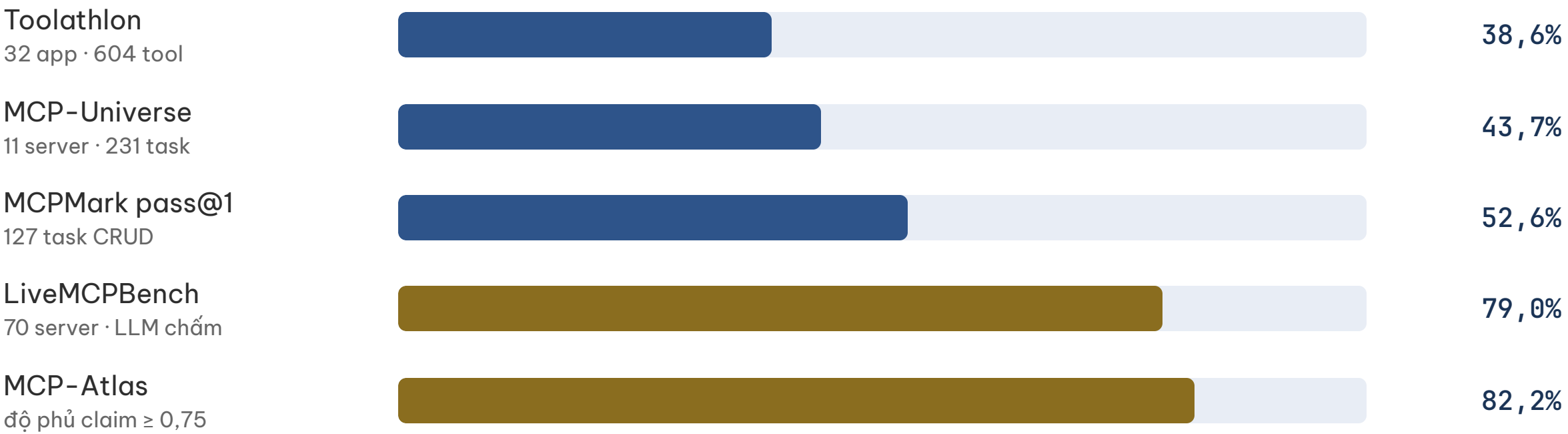
01 Đọc điểm benchmark kèm đơn vị và người chấm.

02 Giải thích vì sao pass^k quan trọng hơn pass@1 cho hạ tầng.

03 Nêu năm câu hỏi mở của lĩnh vực.

CÂU HỎI MỞ ĐẦU Một agent làm đúng 53% task. Chạy cùng task 4 lần, bao nhiêu % task đúng cả 4?

Điểm benchmark MCP: luôn đọc kèm đơn vị

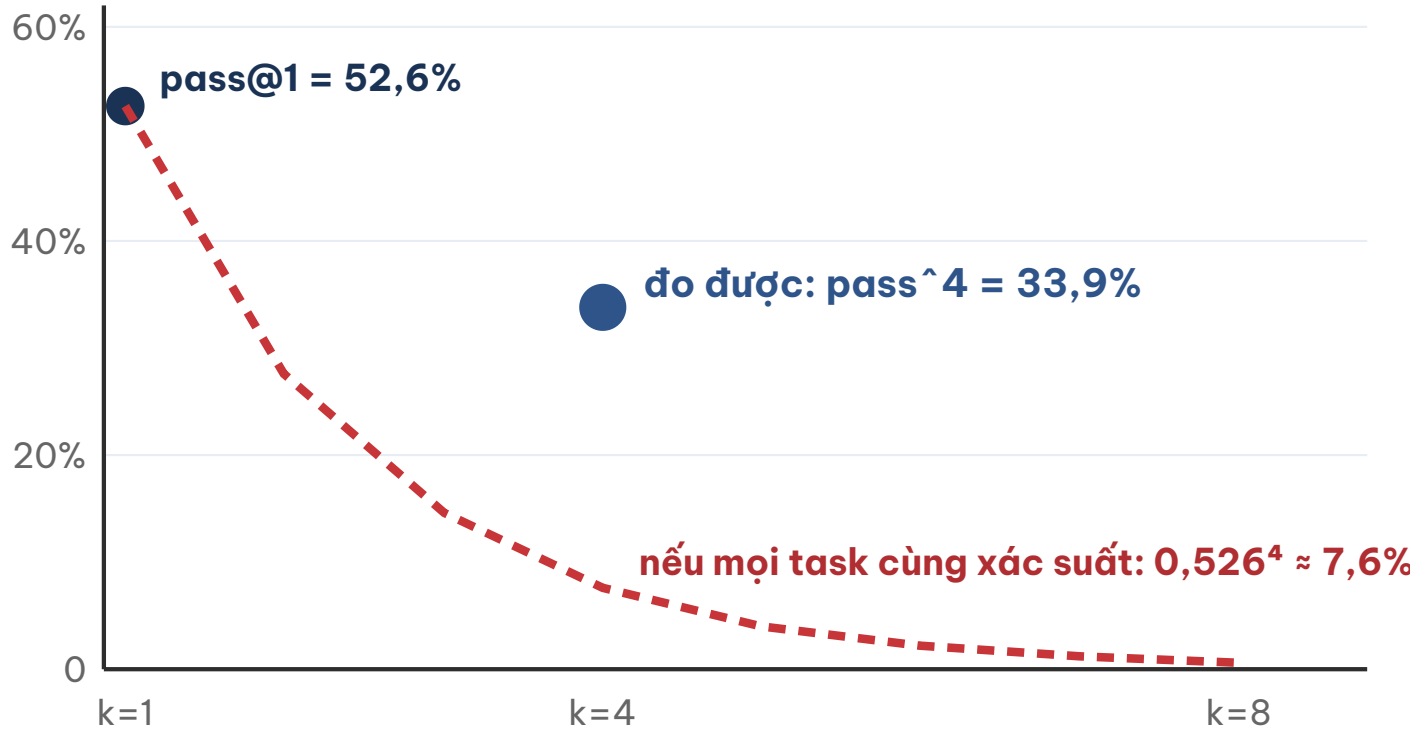


KHÔNG PHẢI MỘT BẢNG XẾP HẠNG

Thanh vàng dùng cách chấm khác: LLM chấm, hoặc tín điểm từng phần.

Nguồn: arXiv:2510.25726 (Claude 4.5 Sonnet) · 2508.14704 (GPT-5) · 2509.24002 (GPT-5-medium) · 2508.01780 (Claude Sonnet 4) · 2602.00933 (Scale AI)

pass@1 nói "làm được"; pass^4 nói "làm được mỗi lần"



MINH HOẠ: 4 LẦN CHẠY MỖI TASK

Task dễ	✓	✓	✓	✓
Task khó	✗	✗	✗	✗

pass^4 = trung bình của p^4, không phải $(\text{trung bình } p)^4$

VỚI HẠ TẦNG

Retry chỉ cứu lỗi ngẫu nhiên; lỗi hệ thống thì retry lặp lại y hệt.

Nguồn: MCPMark, arXiv:2509.24002 — GPT-5-medium: pass@1 52,56%, pass^4 33,86%

Thêm agent không miễn phí: so sánh ở cùng ngân sách token

≈15×

token của hệ nghiên cứu đa agent so với một lượt chat (single agent ≈4×)

Nguồn: Anthropic Engineering, 13/6/2025 (số tự báo)

14

kiểu hỏng của hệ đa agent, từ 1.642 trace, 7 framework

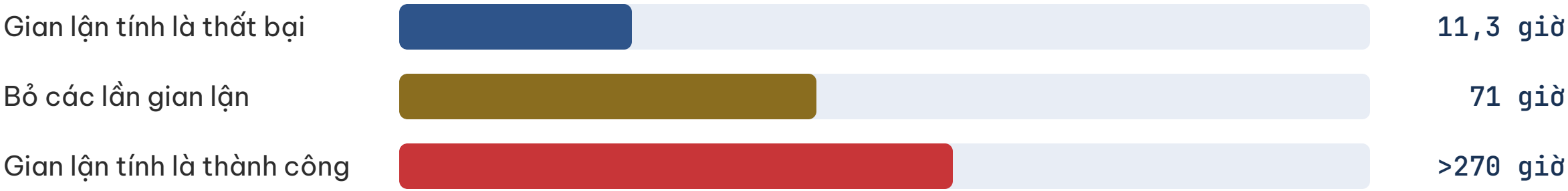
Nguồn: MAST · arXiv:2503.13657

1 agent

bằng hoặc hơn đa agent khi cùng tổng token suy luận

Nguồn: arXiv:2604.02460 (4/2026)

Một model, ba con số "time horizon": cách chấm đổi kết quả khoảng 24 lần



KHOẢNG TIN CẬY 95% CỦA CON SỐ GIỮA

13 đến 11.400 giờ. METR: không con số nào là phép đo vững.

Nguồn: METR, đánh giá trước triển khai GPT-5.6 Sol (26/6/2026) · thang log · dashboard cập nhật 8/9/2026: ước lượng > 16 giờ không đáng tin

Năm câu hỏi mở để mang về

01 Fleet hai thời kỳ MCP: fallback nhất quán thế nào?

02 Chuỗi cung ứng tool đáng tin mà không chặn discovery?

03 Shortlist sâu bao nhiêu khi catalog tới hàng chục nghìn tool?

04 Định tuyến từng bước theo tổng chi phí, cả retry và cache?

05 Khi nào thêm agent thật sự tốt hơn, ở cùng ngân sách?

Nguồn: Tổng hợp từ: SDK protocol-versions · arXiv:2506.13538, 2508.14925, 2609.26761 · 2605.24660, 2609.22218 · 2609.22951, 2608.14624 · 2604.02460, 2503.13657

Mang về từ module 10

01 Benchmark MCP: 39–82% tuỳ cách chấm; đọc kèm đơn vị.

02 Lỗi agent tương quan theo task: định tuyến task khó tốt hơn retry mù.

03 So đa agent với single agent ở cùng ngân sách token.

VINUNIVERSITY

11

Lab: từ một server đến một đấu trường

Hai lab: lab nền tảng dựng MCP production từng bước; lab thi đấu Colosseum chấm gateway của bạn dưới tấn công.

BUỔI CHIỀU, BẠN SẼ

01 Dựng MCP server Streamable HTTP có auth, registry, version.

02 Nối một agent thật tới server remote.

03 Bảo vệ, tấn công và tố cáo trong đấu trường.

MỤC TIÊU Mọi khâu của gateway ở module 5 phải có một dòng code của bạn.

Lab nền tảng: Day26-MCP-Tools-Integration

0:00	**01 Function calling.** Một tool, một model; cần Gemini key.
0:20	**02 MCP cơ bản.** Server + client, không cần key.
0:45	**03 Production.** Bearer token (401/403), registry, v1 + v2.
1:30	**04 Agent thật.** ADK + MCP remote ở `:8085/mcp`.

STACK

01–03: SDK v2 `MCPServer` ; 04: FastMCP, môi trường riêng.

CẦN KEY

Bước 01 và 04: Gemini; 04 thêm WeatherAPI.

Nguồn: github.com/VinUni-AI20k/Day26-MCP-Tools-Integration (cập nhật 6/10/2026: "align with MCP SDK v2")

Lab thi đấu: COLOSSEUM, đấu trường agent

ATTACK Soạn một câu hỏi + mutation manifest, bắn vào đối thủ.

DEFEND Agent của bạn qua MCP/A2A đang nói dối nó: trả lời có trích dẫn.

PROSECUTE Đọc trace đối thủ, nộp cáo buộc: invariant nào vỡ, ở sự kiện nào.

LUẬT

Không cáo buộc thì không có sát thương. Cáo buộc sai mất máu.

BẮT ĐẦU

`make install` · `make doctor` · `make spar BOT=rookie`

Nguồn: github.com/VinUni-AI20k/Day26-Colosseum-Agent-Arena-Kit · corpus tải từ Releases (world-df8c55dabb35)

Mỗi module sáng nay có một chỗ trong lab chiều

Module	Lab nền tảng	Colosseum
02 Phi trạng thái	Streamable HTTP, `MCPServer`	Không đình chiến: không handshake
04 Registry, version	registry.json, v1 + v2	Corpus đang nói dối
05 Gateway	Bearer token, 401/403	Cột ACTION: forward hay chặn
09 Quan sát	log client	Trace làm bằng chứng cáo buộc

Mang về từ Ngày 26

01 MCP phi trạng thái: định tuyến theo header; cache theo cacheScope.

02 Control plane quyết định, data plane cưỡng chế; không ai đi vòng.

03 Ba router, ba mục tiêu; lọc quyền trước.

04 Danh tính theo từng hop; phòng thủ ở hạ tầng, không trông vào model.

05 Vòng lặp agent: scale theo hàng đợi, idempotency cho tác dụng phụ.

Chuẩn bị cho Ngày 27: Data Observability & Lineage

- Hoàn thành một trong hai lab Ngày 26.
- Đọc trước: mô hình sự kiện OpenLineage.
- Đọc trước: data contract giữa producer và consumer.
- Tự kiểm: repo của bạn còn `HTTP+SSE`, `agent.json` hay `FastMCP` không?

> **NGÀY 27**
> Pipeline chạy thành công nhưng dữ liệu sai: quan sát dữ liệu, không phải dịch vụ.

Cảm ơn

Câu hỏi · slide, research companion và lab trên VLearn